
第2章

布尔运算与**CMOS**逻辑门

Boolean Operation & CMOS Logic Gates

参考：讲义第2章

2020年9月15日

布尔运算与CMOS逻辑门

- 二进制数和二进制逻辑
- 布尔代数和布尔函数
- 范式与标准式
- MOSFET开关
- 基本的CMOS逻辑门
- 门电路化简
- CMOS复合逻辑门

二进制数

■ 十进制数

- 位置记数的系数序列，如：**7392**
- 基数为**10**
- 系数**0~9**
- 例如：

$$7392 = 7 \times 10^3 + 3 \times 10^2 + 9 \times 10^1 + 2 \times 10^0$$

$$a_5 a_4 a_3 a_2 a_1 a_0 . a_{-1} a_{-2} a_{-3}$$

$$10^5 a_5 + 10^4 a_4 + 10^3 a_3 + 10^2 a_2 + 10^1 a_1 + 10^0 a_0 \\ + 10^{-1} a_{-1} + 10^{-2} a_{-2} + 10^{-3} a_{-3}$$

$$a_5, a_4, a_3, a_2, a_1, a_0, a_{-1}, a_{-2}, a_{-3} = 0 \sim 9$$

二进制数

■ 二进制数

- 基数为2
- 系数0, 1
- 如: **11010.11**

$$1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} \\ = \mathbf{26.75}$$

二进制数

■ 其它进制

- 基数为 r
- 系数 $0 \sim r-1$
- 如：八进制、十六进制

$$a_n \cdot r^n + a_{n-1} \cdot r^{n-1} + \dots \\ + a_1 \cdot r^1 + a_0 \cdot r^0 + a_{-1} \cdot r^{-1} + a_{-2} \cdot r^{-2} + \dots + a_{-m} \cdot r^{-m}$$

$$(127.4)_8 = 1 \times 8^2 + 2 \times 8^1 + 7 \times 8^0 + 4 \times 8^{-1} = (87.5)_{10}$$

$$(B65F)_{16} = 11 \times 16^3 + 6 \times 16^2 + 5 \times 16^1 + 15 \times 16^0 \\ = (46687)_{10}$$

二进制数

■ 二进制数转换

- 二进制数又称为比特数，将一个二进制数转化为十进制数时，把比特数为**1**的那些位置所对应的**2**的幂数相加

$$(110101)_2 = 2^5 + 2^4 + 2^2 + 2^0 = (53)_{10}$$

- **2的n次幂所对应的数：**
 - ◆ $2^{10} = 1,024$ (1K)
 - ◆ $2^{20} = 1,048,576$ (1M)
 - ◆ $2^{30} = 1,073,741,824$ (1G)
 - ◆ $2^{40} = 1,099,511,627,776$ (1T)

二进制数

■ 十进制数转换为二进制数

- 整数部分和小数部分分别进行转换
- 整数部分除以 r ，取余数；所得之商再除以 r ，取余数；...
- 例如十进制数**41**转换成二进制数

商数	余数	系数
41		
20	1	$a_0 = 1$
10	0	$a_1 = 0$
5	0	$a_2 = 0$
2	1	$a_3 = 1$
1	0	$a_4 = 0$
0	1	$a_5 = 1$

$$\begin{aligned}(41)_{10} &= a_5 a_4 a_3 a_2 a_1 a_0 \\ &= (101001)_2\end{aligned}$$

二进制数

■ 十进制数转换为二进制数

- 整数部分和小数部分分别进行转换
- 十进制小数乘以 r ，得到一个整数和一个小数；新的小数再乘以 r ，又得到一个整数和一个小数；...
- 例如十进制数**0.6875**转换成二进制数

	余数	系数
$0.6875 \times 2 =$	1.3750	$a_{-1} = 1$
$0.3750 \times 2 =$	0.7500	$a_{-2} = 0$
$0.7500 \times 2 =$	1.5000	$a_{-3} = 1$
$0.5000 \times 2 =$	1.0000	$a_{-4} = 1$

$$(0.6875)_{10} = (0.a_{-1}a_{-2}a_{-3}a_{-4})_2 = (0.1011)_2$$

二进制数

■ 十进制数转换为八进制数

● 十进制数**153**转换成八进制数

商数	余数	系数
153		
19	1	$a_0 = 1$
2	3	$a_1 = 3$
0	2	$a_2 = 2$

$$(153)_{10} = a_2 a_1 a_0 = (231)_8$$

二进制数

■ 十进制数转换为八进制数

- 例如十进制数**0.513**转换成八进制数

	余数	系数
$0.513 \times 8 =$	4.104	$a_{-1} = 4$
$0.104 \times 8 =$	0.832	$a_{-2} = 0$
$0.832 \times 8 =$	6.656	$a_{-3} = 6$
$0.656 \times 8 =$	5.248	$a_{-4} = 5$
$0.248 \times 8 =$	1.984	$a_{-5} = 1$
$0.984 \times 8 =$	7.872	$a_{-6} = 7$

$$(0.513)_{10} = (0.a_{-1}a_{-2}a_{-3}a_{-4}a_{-5}a_{-6}\cdots)_8 = (0.406517\cdots)_8$$

二进制数

■ 二进制数运算

- r 进制数的算术运算规则与十进制数相同，例如二进制数的加法、减法和乘法

$$\begin{array}{r} \text{被加数:} \quad 101101 \\ \text{加数:} \quad + \quad 100111 \\ \hline \text{和:} \quad \quad 1010100 \end{array}$$

$$\begin{array}{r} \text{被减数:} \quad 101101 \\ \text{减数:} \quad - \quad 100111 \\ \hline \text{差:} \quad \quad 000110 \end{array}$$

$$\begin{array}{r} \text{被乘数:} \quad 1011 \\ \text{乘数:} \quad \times \quad 101 \\ \hline \quad \quad 1011 \\ \quad 0000 \\ \quad 1011 \\ \hline \text{积:} \quad \quad 110111 \end{array}$$

八进制与十六进制

- 每个八进制数位对应**3个**二进制数位；每个十六进制数位对应**4个**二进制数位
- 二进制数转换为八进制数：从小数点开始，**分别向左、向右进行**，**每3个**数位分为一组
 $(\underline{10} \ \underline{110} \ \underline{001} \ \underline{101} \ \underline{011} . \underline{111} \ \underline{100} \ \underline{000} \ \underline{110})_2 = (26153.7406)_8$
- 二进制数转换为十六进制数：与转换为八进制数类似，**每4个**二进制数位分为一组
 $(\underline{10} \ \underline{1100} \ \underline{0110} \ \underline{1011} . \underline{1111} \ \underline{0000} \ \underline{0110})_2 = (2C6B.F06)_{16}$
- 八进制数或十六进制数转换为二进制数的步骤刚好与上述步骤相反，每个八进制/十六进制数位转换为**3/4个**二进制数位

将以下二进制数转换为十进制数： (a) 1.10010;
(b) 110.010

答： (a) [填空1] (b) [填空2]

正常使用填空题需3.0以上版本雨课堂

用8 位二进制数表示十进制的 $2/3$ ，结果是： [填空1]

再将这8位二进制数转换为十进制数，结果是： [填空2]

正常使用填空题需3.0以上版本雨课堂

二进制逻辑

■ 二进制逻辑处理具有两个离散值的变量和具有某种逻辑意义的运算

- 变量的两个值：真/假，是/非，0/1

■ 二进制逻辑定义

- 二进制逻辑包括二进制逻辑变量和一组逻辑运算

- ◆ 逻辑变量一般用**A**、**B**、**C**、**x**、**y**、**z**等字母表示，每个逻辑变量只能有**0**和**1**两种可能的取值
- ◆ 基本逻辑运算有：“与”、“或”、“非”
- ◆ “与”： $x \cdot y = z$ ，或 $xy = z$ 。当且仅当**x**和**y**同时为**1**时，才有**z=1**；否则**z=0**
- ◆ “或”： $x + y = z$ 。当**x=1**或**y=1**时，**z=1**；否则，**z=0**
- ◆ “非”： $x' = z$ ，或 $\bar{x} = z$ 。若**x=1**，则**z=0**；若**x=0**，则**z=1**

二进制逻辑

■ 逻辑运算真值表

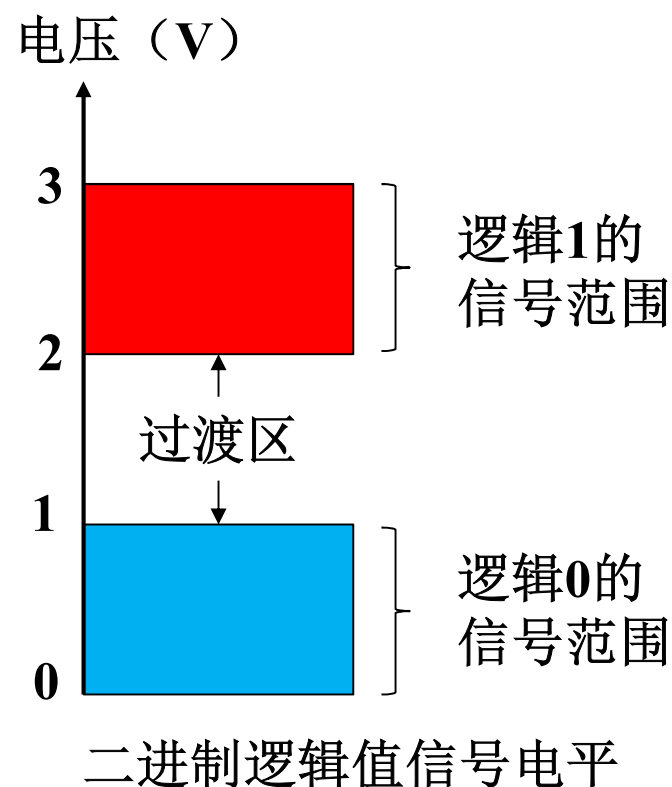
- 对于 x 和 y 逻辑取值的每一种组合，都有由逻辑运算所定义的 z 值与之对应，可以用**真值表**的方式列出
- **真值表**是一种将输入变量的各种可能取值组合，与相应运算结果一起列成的表

与			或			非	
x	y	$x \cdot y$	x	y	$x + y$	x	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

二进制逻辑

■ 模拟信号与逻辑值的映射

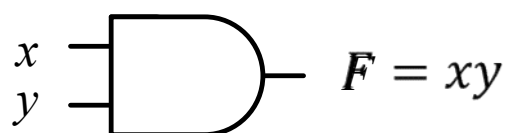
- 模拟信号的电压范围**0~3V**
- 数字系统中变成两种可识别的逻辑值**0**或**1**：逻辑**0**（逻辑低电平）对应于**0V**电压信号，逻辑**1**（逻辑高电平）对应于**3V**电压信号
- 每个电压电平都有一个可接受的范围，如右图所示



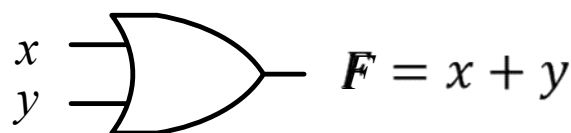
二进制逻辑

■ 逻辑门

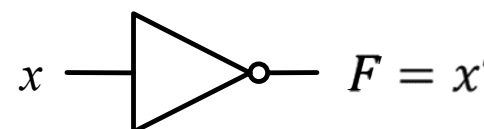
- 逻辑门是一个或多个输入信号进行运算，并产生输出信号的电子电路，与、或、非三种逻辑门的图形符号



二输入与门 (AND)



二输入或门 (OR)



非门/反相器 (INV)

- 当输入逻辑满足一定的要求时，产生相当于逻辑**1**或逻辑**0**的输出信号
- 与门和或门的输入信号可以超过两个，如三输入与门、四输入或门等

布尔运算与CMOS逻辑门

- 二进制数和二进制逻辑
- 布尔代数和布尔函数
- 范式与标准式
- MOSFET开关
- 基本的CMOS逻辑门
- 门电路化简
- CMOS复合逻辑门

布尔代数

- 布尔代数是处理**二进制变量**和**逻辑运算**的代数。布尔代数可以用一组元素、运算和不需要证明的公理或假设来定义
- 布尔代数的公理化定义
 - 1854年，乔治·布尔提出了逻辑的系统化处理，并发展了一个代数系统
 - 1938年，香农提出了一种称为开关代数的**二值布尔代数**，并论证了双稳态电子开关电路可以用这种代数来表示
- 布尔代数是一个由元素集 B 与两个二元运算符“+”和“ $\cdot$ ”来定义的代数结构，满足下面的这些公理（由E. V. Huntington提出）：

布尔代数

1. 对运算符“+”和“ $\bullet$ ”，结构是封闭的
2. (a) “+”运算的单位元是0，满足： $x + 0 = 0 + x = x$
(b) “ $\bullet$ ”运算的单位元是1，满足： $x \bullet 1 = 1 \bullet x = x$
3. “+”和“ $\bullet$ ”运算满足交换律：
$$x + y = y + x, \quad x \bullet y = y \bullet x$$
4. (a) “ $\bullet$ ”运算对“+”运算的分配律为：
$$x \bullet (y + z) = (x \bullet y) + (x \bullet z)$$

(b) “+”运算对“ $\bullet$ ”运算的分配律为：
$$x + (y \bullet z) = (x + y) \bullet (x + z)$$
5. 对于每一个元素 $x \in B$ ，都存在一个元素 $x' \in B$ ，满足：
(a) $x + x' = 1$ ； (b) $x \bullet x' = 0$
6. 至少存在两个元素 $x, y \in B$ ，有 $x \neq y$

布尔代数

- 构造一个布尔代数，一定要有：
 - 集合 B 中的元素
 - 两个二元运算符的运算法则
 - 元素集 B 与两个运算符满足6条Huntington公理
- 选择不同的元素集和运算法则，可以设计出多种布尔代数
- 二值布尔代数：元素集中仅有两个元素

二值布尔代数

- 二值布尔代数定义在一个两元素集 $B = \{0, 1\}$ 上，其两个二元运算符 $+$ 和 $\cdot$ 的运算法则如下：

x	y	$x \cdot y$	x	y	$x + y$	x	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

- 可以验证，对集合 $B = \{0, 1\}$ 和两个二元运算符，**Huntington**公理均成立

布尔代数的基本定理和性质

■ 对偶性

- **Huntington**假设成对地列出：部分**(a)**和部分**(b)**。若将二进制运算符和单位元互换，即可由部分**(a)**得出部分**(b)**，反之亦然。布尔代数的这个重要性质称为对偶原理
- 若要找出一个代数表达式的“对偶式”，只要简单互换“或”和“与”运算符，并且将**0**变**1**、**1**变**0**即可

布尔代数的基本定理和性质

■ 基本定理

表2.4 布尔代数的公理和定理

公理 2	(a) $x + 0 = x$	(b) $x \cdot 1 = x$
公理 5	(a) $x + x' = 1$	(b) $x \cdot x' = 0$
定理 1	(a) $x + x = x$	(b) $x \cdot x = x$
定理 2	(a) $x + 1 = 1$	(b) $x \cdot 0 = 0$
定理 3, 对合性	$(x')' = x$	
公理 3, 交换性	(a) $x + y = y + x$	(b) $x y = y x$
定理 4, 结合性	(a) $x + (y + z) = (x + y) + z$	(b) $x(yz) = (xy)z$
公理 4, 分配性	(a) $x(y + z) = xy + xz$	(b) $x + yz = (x + y)(x + z)$
定理 5, 摩根定理	(a) $(x + y)' = x'y'$	(b) $(xy)' = x' + y'$
定理 6, 吸收性	(a) $x + xy = x$	(b) $x(x + y) = x$

布尔代数的基本定理和性质

■ 基本定理

- 定理证明（略）

■ 运算符优先级（从高到低）

- 圆括号、“非”、“与”、“或”

布尔函数

- 布尔函数用二进制变量、常量**0**和**1**以及逻辑运算符组成的一个代数表达式来描述

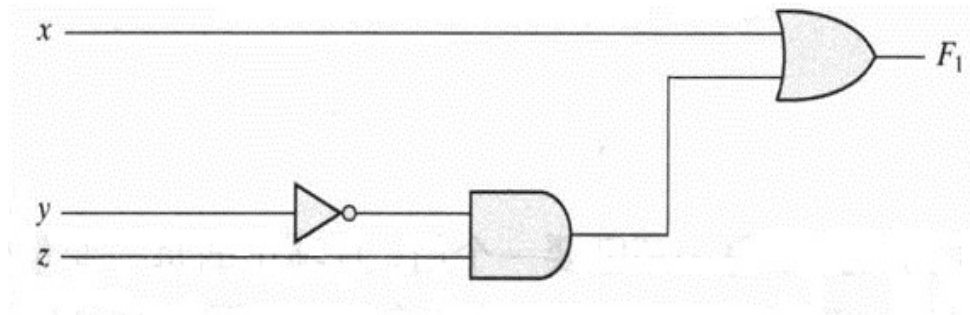
$$F_1 = x + y'z$$

- 布尔函数表示二进制变量间的逻辑关系，它的值由二进制表达式的值来确定
- 布尔函数可以用真值表来表示
 - 真值表是将二进制变量赋值为**0**、**1**时的各种组合情况，及其相应的函数值列成的表

x	y	z	F_1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

布尔函数

- 布尔函数可以从代数表达式转化为由逻辑门构成的电路图



$$F_1 = x + y'z$$

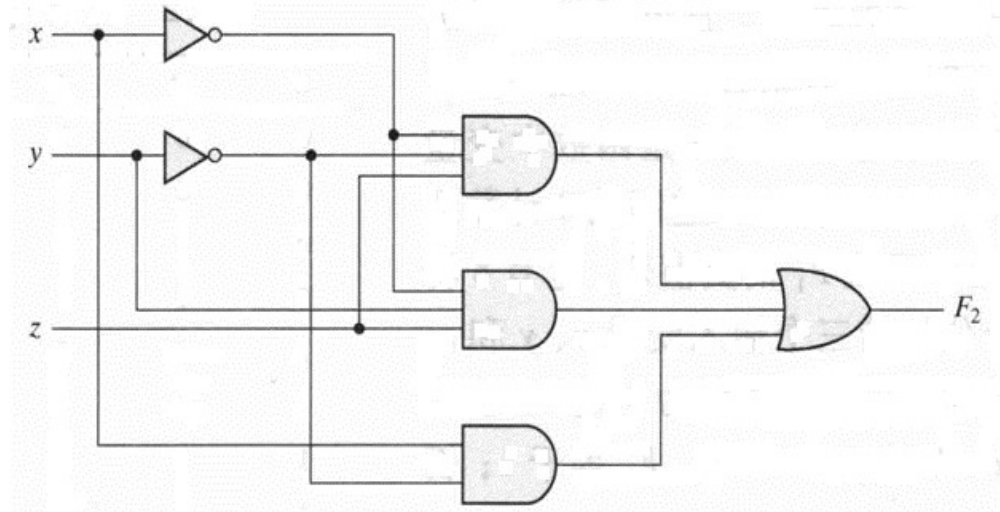
- 用代数方式来表示布尔函数时，可以有多种不同的表示形式

布尔函数

■ 例如布尔函数:

$$F_2 = x'y'z + x'yz + xy'$$

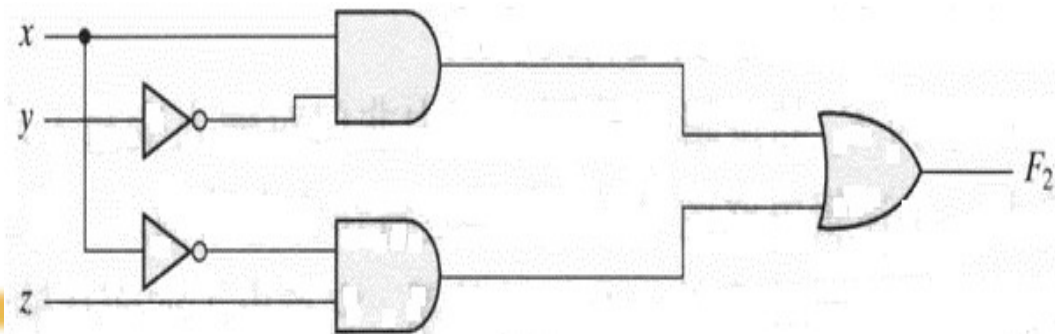
函数的逻辑门实现



■ 运用布尔代数性质对函数化简

$$F_2 = x'y'z + x'yz + xy' = x'z(y' + y) + xy' = x'z + xy'$$

逻辑门实现



布尔函数

■ 代数变换

- 对布尔表达式进行化简，以获得一个更简单的电路
- 将布尔函数简化到最少的字母数

1. $x(x' + y) = xx' + xy = 0 + xy = xy$

2. $x + x'y = (x + x')(x + y) = 1(x + y) = x + y$

3. $(x + y)(x + y') = x + xy + xy' + yy' = x(1 + y + y') = x$

4. $xy + x'z + yz = xy + x'z + yz(x + x') = xy + x'z + xyz + x'yz = xy(1 + z) + x'z(1 + y) = xy + x'z$

5. $(x + y)(x' + z)(y + z) = (x + y)(x' + z)$

(函数4的对偶式)

布尔函数

- 反函数：对函数 F 的取反，记为 F'
 - 反函数可以利用摩根定理用代数法推导而得
 - 两变量的摩根定理见表2.4
 - 三变量摩根定理：

$$\begin{aligned}(A + B + C)' &= (A + x)' && \text{设 } B + C = x \\ &= A'x' && \text{定理5(a) (摩根定理)} \\ &= A'(B + C)' \\ &= A'(B'C') && \text{摩根定理} \\ &= A'B'C' && \text{定理4(b) (结合律)}\end{aligned}$$

布尔函数

■ 反函数

- 摩根定理的推广形式:

$$(A + B + C + D + \dots + F)' = A'B'C'D'\dots F'$$

$$(ABCD\dots F)' = A' + B' + C' + D' + \dots + F'$$

反函数就是将“与”运算符和“或”运算符互换，并将每个变量取反而得

- 求函数 $F_1 = x'yz' + x'y'z$ 与 $F_2 = x(y'z' + yz)$ 的反函数

$$F_1' = (x'yz' + x'y'z)' = (x'yz')'(x'y'z)'$$

$$= (x+y'+z)(x+y+z')$$

$$F_2' = (x(y'z' + yz))' = x' + (y'z' + yz)'$$

$$= x' + (y'z')'(yz)' = x' + (y+z)(y'+z')$$

$$= x' + yz' + y'z$$

布尔运算与CMOS逻辑门

- 二进制数和二进制逻辑
- 布尔代数和布尔函数
- 范式与标准式
- MOSFET开关
- 基本的CMOS逻辑门
- 门电路化简
- CMOS复合逻辑门

范式和标准式

■ 最小项和最大项

● 最小项（标准积项）

- ◆ n 个变量组成的“与”项
- ◆ n 个变量可以列出0至 2^n-1 个二进制数，若二进制数的相应位为0，则取反变量；若二进制数的相应位为1，则取原变量

● 最大项（标准和项）

- ◆ n 个变量组成的“或”项
- ◆ n 个变量有 2^n 个最大项，若二进制数的相应位为0，则取原变量；若二进制数的相应位为1，则取反变量

范式和标准式

三变量的最小项和最大项

x	y	z	最小项		最大项	
			项	标识	项	标识
0	0	0	$x'y'z'$	m_0	$x+y+z$	M_0
0	0	1	$x'y'z$	m_1	$x+y+z'$	M_1
0	1	0	$x'yz'$	m_2	$x+y'+z$	M_2
0	1	1	$x'yz$	m_3	$x+y'+z'$	M_3
1	0	0	$xy'z'$	m_4	$x'+y+z$	M_4
1	0	1	$xy'z$	m_5	$x'+y+z'$	M_5
1	1	0	xyz'	m_6	$x'+y'+z$	M_6
1	1	1	xyz	m_7	$x'+y'+z'$	M_7

每一个最大项就是其相应的最小项的反函数，反之亦然

范式和标准式

■ 最小项和最大项

- 任何一个布尔函数可表示为最小项之和（各个项的或）

真值表中函数值为**1**的每个变量组合形成一个最小项，再将所有这些最小项相**或**

- 任何一个布尔函数可表示为最大项之积（各个项的与）

真值表中函数值为**0**的每个变量组合形成一个最大项，再将所有这些最大项相**与**

将布尔函数表示成最小项之和或最大项之积，称为布尔函数的**标准形式**

范式和标准式

■ 用最小项之和表示布尔函数

- 先把表达式扩展成与项之和，然后再检查每项是否包含所有变量。如果缺1个或更多的变量，可用 $x+x'$ 的表达式与之相与（ x 为所缺变量）
- 例：用最小项之和来表示布尔函数 $F = A + B'C$

$$\begin{aligned} F &= A + B'C = A(B+B') + (A+A')B'C \\ &= AB(C+C') + AB'(C+C') + AB'C + A'B'C \\ &= ABC + ABC' + AB'C + AB'C' + A'B'C \\ &= A'B'C + AB'C' + AB'C + ABC' + ABC \\ &= m_1 + m_4 + m_5 + m_6 + m_7 \end{aligned}$$

$$F(A, B, C) = \sum (1, 4, 5, 6, 7)$$

范式和标准式

■ 用最大项之积表示布尔函数

- 先把表达式变成或项之积，通过分配律 $x+yz = (x+y)(x+z)$ 来实现
- 对于缺变量的或项，可用 xx' 与之相或（ x 为所缺变量）
- 例：用最大项之积来表示布尔函数 $F = xy + x'z$

$$\begin{aligned} F &= xy + x'z = (xy+x')(xy+z) = (x+x')(y+x')(x+z)(y+z) \\ &= (y+x'+zz')(x+z+yy')(y+z+xx') \\ &= (x'+y+z)(x'+y+z')(x+y+z)(x+y'+z)(x+y+z)(x'+y+z) \\ &= (x+y+z)(x+y'+z)(x'+y+z)(x'+y+z') \\ &= M_0 M_2 M_4 M_5 \end{aligned}$$

$$F(x, y, z) = \prod (0, 2, 4, 5)$$

范式和标准式

■ 标准形式间的转换

- 用最小项之和表示的函数的反函数

$$F(A, B, C) = \sum (1, 4, 5, 6, 7)$$

$$F'(A, B, C) = \sum (0, 2, 3) = m_0 + m_2 + m_3$$

用摩根定理对 F' 取反:

$$\begin{aligned} F &= (m_0 + m_2 + m_3)' = m'_0 m'_2 m'_3 = M_0 M_2 M_3 \\ &= \prod (0, 2, 3) \end{aligned}$$

从一种标准形式转化为另一种标准形式时，互换求和号 Σ 与连乘号 Π ，列出原形式中所缺的项数。

范式和标准式

■ 标准形式间的转换

- 用真值表和标准转化方法，将布尔函数从逻辑表达式转化为最大项之积

$$F = xy + x'z$$

函数真值表

最小项之和表示：

$$F(x, y, z) = \sum (1, 3, 6, 7)$$

最大项之积表示：

$$F(x, y, z) = \prod (0, 2, 4, 5)$$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

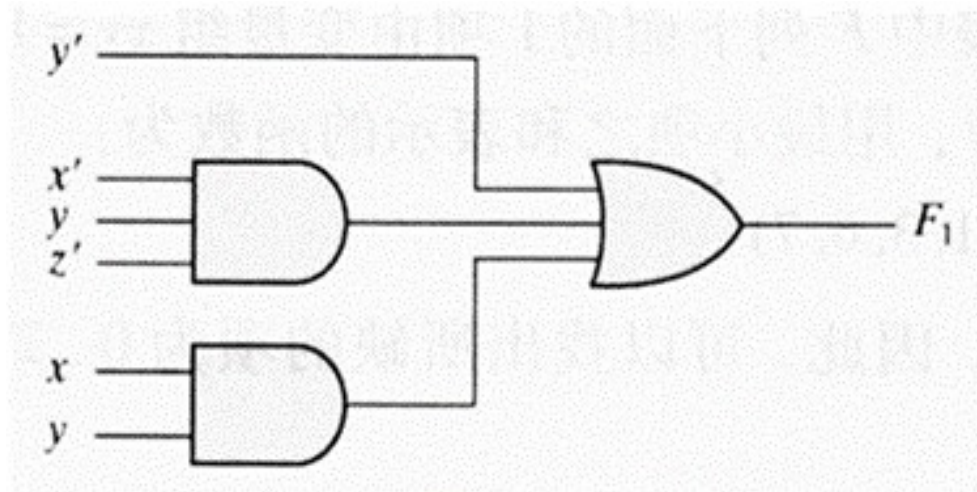
范式和标准式

■ 标准式

- 函数项可以包含一个、两个或任意个字母
- 两种类型的标准形式：积之和式、和之积式
- 积之和式：包含多个与项（乘积项）的布尔表达式

$$F_1 = y' + xy + x'yz'$$

积之和式的逻辑图由一组与门和一个或门组成，二级门电路结构



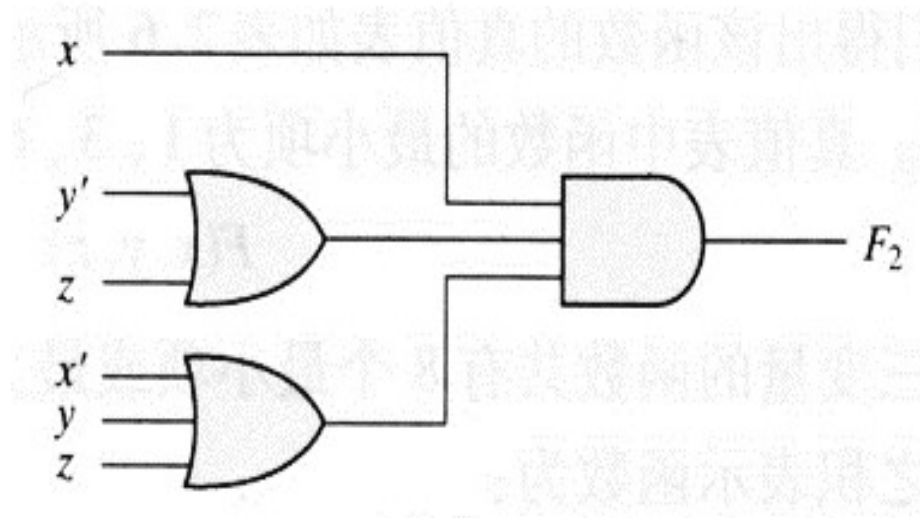
范式和标准式

■ 标准式

- 和之积式：由或项（求和项）组成的布尔表达式

$$F_2 = x(y' + z)(x' + y + z)$$

和之积式的门电路结构由一组用于求和的或门和一个与门组成，也是一种二级门电路结构



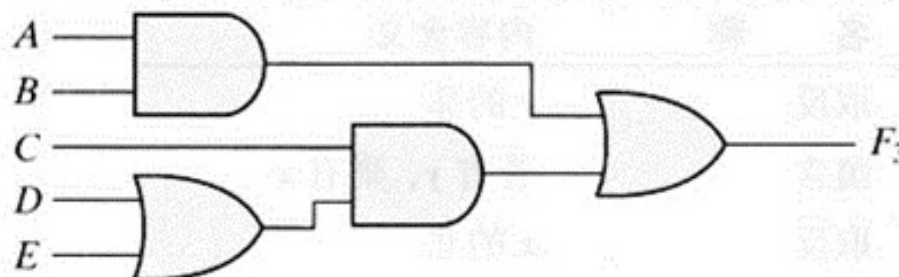
范式和标准式

■ 标准式

- 非标准形式：既不是积之和式，也不是和之积式

$$F_3 = AB + C(D + E)$$

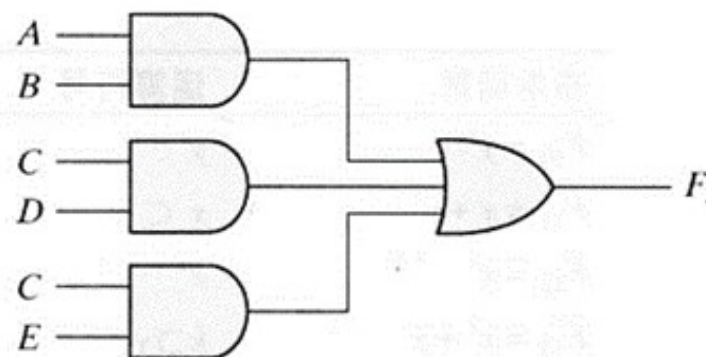
门电路实现需要2个与门2个或门，
电路结构是3级



利用分配律，变换成标准式（积之和式）

$$\begin{aligned} F_3 &= AB + C(D + E) \\ &= AB + CD + CE \end{aligned}$$

门电路实现用3个与门和1个或门，
二级结构



其他逻辑运算

■ n 个变量可以组成 2^{2^n} 个函数： $n=2$ 可以形成**16**个布尔函数

● 两个变量的**16**个函数的真值表

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

其他逻辑运算

- n 个变量布尔函数
- 两个变量

布尔函数	运算符号	名称	内容含义
$F_0 = 0$		零	二进制常数0
$F_1 = xy$	$x \cdot y$	与	x 与 y
$F_2 = xy'$	x/y	禁止	x , 而不是 y
$F_3 = x$		传输	x
$F_4 = x'y$	y/x	禁止	y , 而不是 x
$F_5 = y$		传输	y
$F_6 = xy' + x'y$	$x \oplus y$	异或	x 异或 y
$F_7 = x + y$	$x + y$	或	x 或 y
$F_8 = (x + y)'$	$(x + y)'$ 或 $x \downarrow y$	或非	或非
$F_9 = x'y' + xy$	$(x \oplus y)'$	同或 (异或非)	x 等于 y
$F_{10} = y'$	y'	取反	y 的非
$F_{11} = x + y'$	$x \subset y$	蕴含	若有 y , 则有 x
$F_{12} = x'$	x'	取反	x 的非
$F_{13} = x' + y$	$x \supset y$	蕴含	若有 x , 则有 y
$F_{14} = (xy)'$	$x \uparrow y$	与非	与非
$F_{15} = 1$		等同	二进制常数1

其他逻辑运算

■ n 个变量可以组成 2^{2^n} 个函数： $n=2$ 可以形成16个布尔函数

● 16个函数分为三类：

◆ 2个函数产生常量0或1

◆ 4个函数为一元运算：取反和传输

◆ 10个函数具有二元运算符，定义了8种不同的运算：“与”、“或”、“与非”、“或非”、“异或”、“同或”、“禁止”、“蕴含”

◆ 常用逻辑门符号：NOR、NAND、OR、AND、XOR、XNOR

其他逻辑运算

■ n -
尔

布尔函数	运算符号	名称	内容含义
$F_0 = 0$		零	二进制常数0
$F_1 = xy$	$x \cdot y$	与	x 与 y
$F_2 = xy'$	x/y	禁止	x , 而不是 y
$F_3 = x$		传输	x
$F_4 = x'y$	y/x	禁止	y , 而不是 x
$F_5 = y$		传输	y
$F_6 = xy' + x'y$	$x \oplus y$	异或	x 异或 y
$F_7 = x + y$	$x + y$	或	x 或 y
$F_8 = (x + y)'$	$(x + y)'$ 或 $x \downarrow y$	或非	或非
$F_9 = x'y' + xy$	$(x \oplus y)'$	同或 (异或非)	x 等于 y
$F_{10} = y'$	y'	取反	y 的非
$F_{11} = x + y'$	$x \subset y$	蕴含	若有 y , 则有 x
$F_{12} = x'$	x'	取反	x 的非
$F_{13} = x' + y$	$x \supset y$	蕴含	若有 x , 则有 y
$F_{14} = (xy)'$	$x \uparrow y$	与非	与非
$F_{15} = 1$		等同	二进制常数1

数字逻辑门

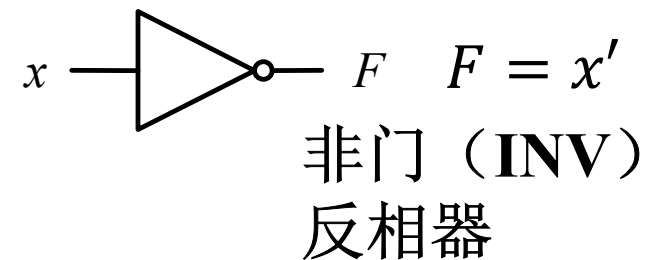
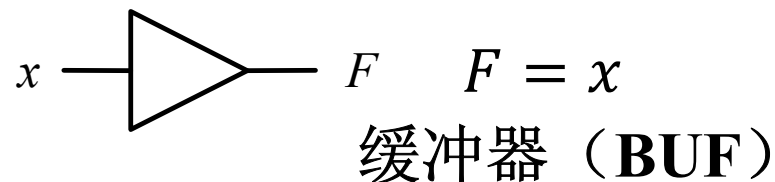
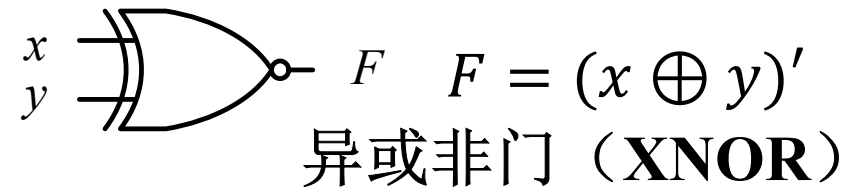
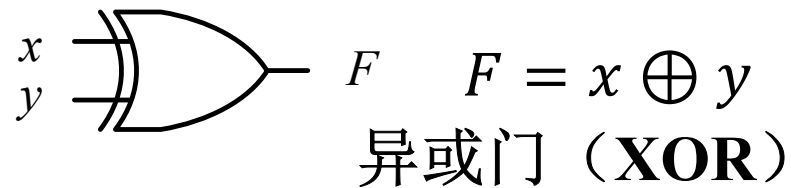
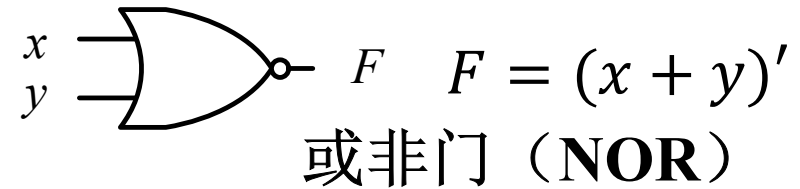
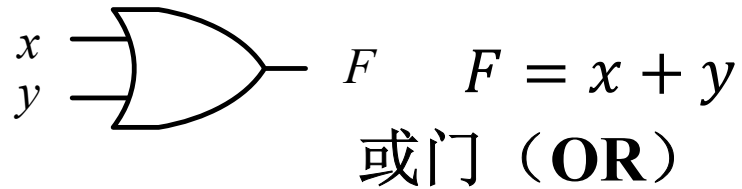
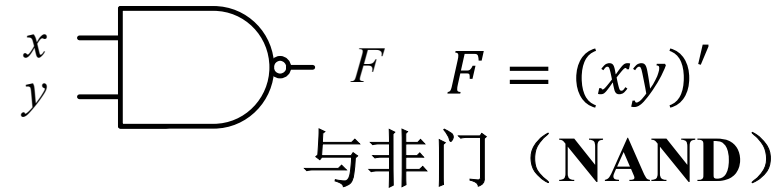
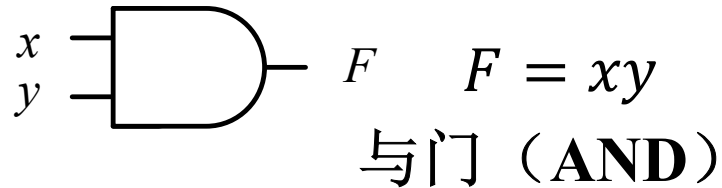
■ 构建逻辑门需要考虑的因素

- 可行性和经济性
- 扩展到2个以上输入的可行性
- 可交换性和可结合性
- 单独实现布尔函数的能力或连接其他逻辑门的能力

■ 8个函数在数字设计中用作标准的逻辑门

- 取反、传输、与、或、与非、或非、异或、同或

数字逻辑门



数字逻辑门

■ 输入扩展

- 如果逻辑门代表的二进制运算具有交换性和结合性，则可以被扩展成多个输入

- ◆ 与运算和或运算：

$$x + y = y + x \quad (\text{交换性})$$

$$(x + y) + z = x + (y + z) = x + y + z \quad (\text{结合性})$$

- ◆ 与非和或非函数：具有交换性

$$(x + y)' = (y + x)' \quad (\text{交换性})$$

$$(x \downarrow y) \downarrow z = ((x + y)' + z)' = (x + y)z' \neq x \downarrow (y \downarrow z)$$

(不具有结合性)

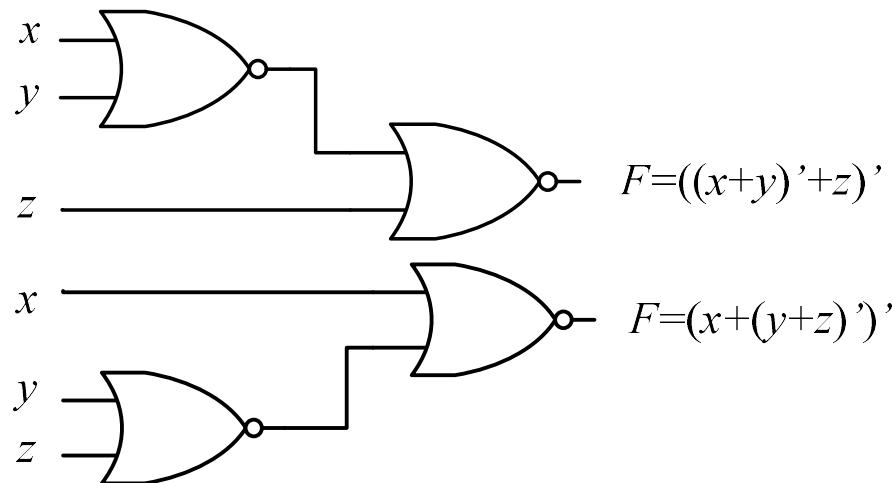
多输入“或非门”（或“与非门”）定义为“或门”（或“与门”）的取反

$$x \downarrow y \downarrow z = (x + y + z)', \quad x \uparrow y \uparrow z = (xyz)'$$

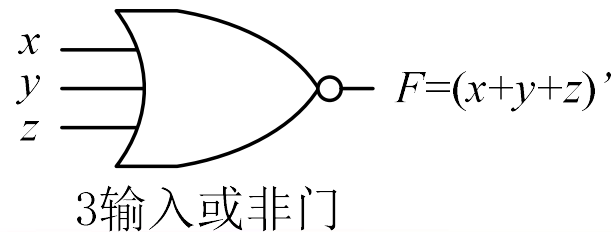
数字逻辑门

■ 输入扩展

- 与非和或非函数：具有交换性，没有可结合性



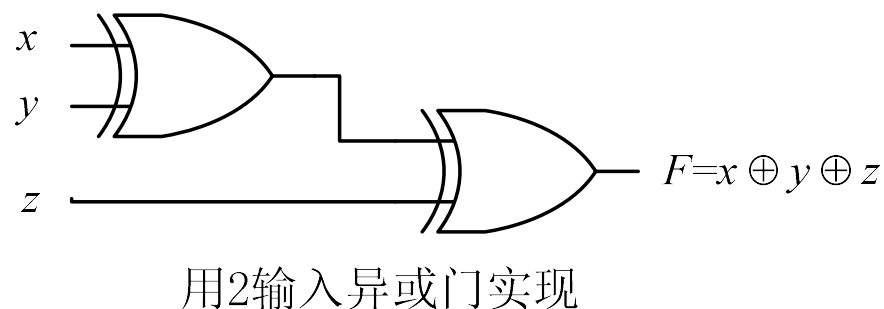
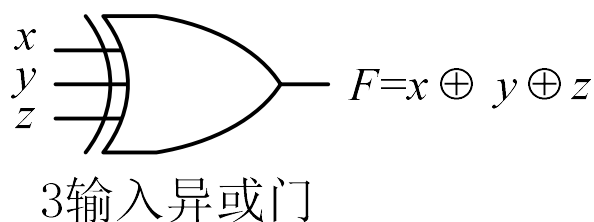
- 多输入或非门/与非门定义为或门/与门的取反



数字逻辑门

■ 输入扩展

- 如果逻辑门代表的二进制运算具有交换性和结合性，则可以被扩展成多个输入
 - ◆ 异或门和同或门：具有交换性和结合性，均可以扩展至2个以上输入
 - ◆ 多输入异或门用硬件实现比较复杂
 - ◆ 扩展至2个以上输入时，异或函数定义相应修改：输入变量有奇数个1时，函数值为1

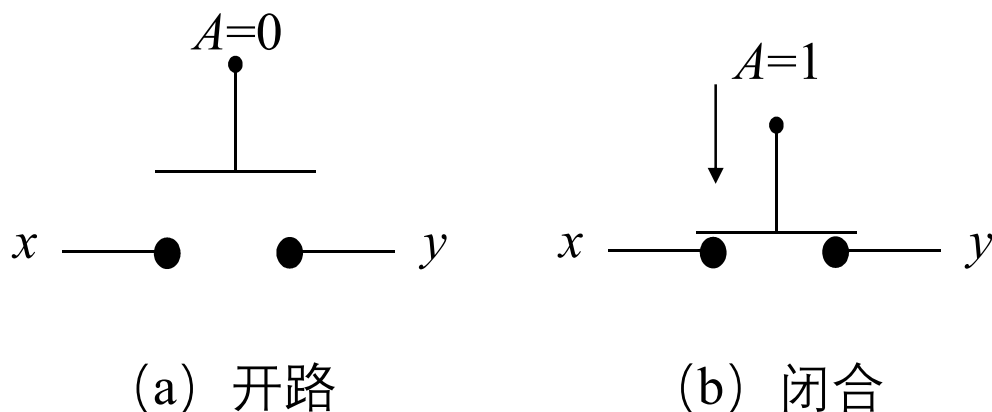


布尔运算与CMOS逻辑门

- 二进制数和二进制逻辑
- 布尔代数和布尔函数
- 范式与标准式
- **MOSFET开关**
- 基本的CMOS逻辑门
- 门电路化简
- CMOS复合逻辑门

布尔运算与理想开关

高电平有效开关

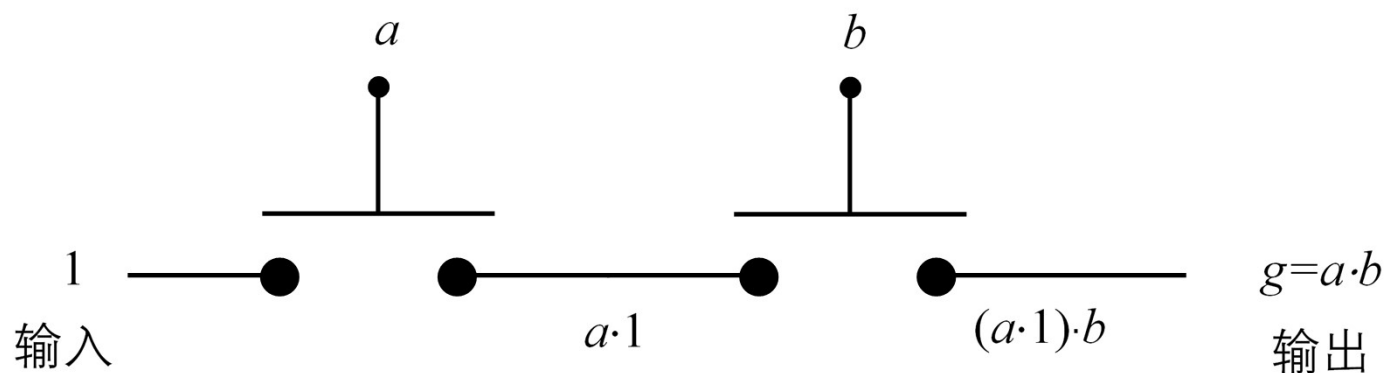


$$y = x \cdot A \quad \text{iff} \quad A = 1$$

当 $A=0$ 时， x 与 y 之间的关系是不确定的

布尔运算与理想开关

高电平有效开关



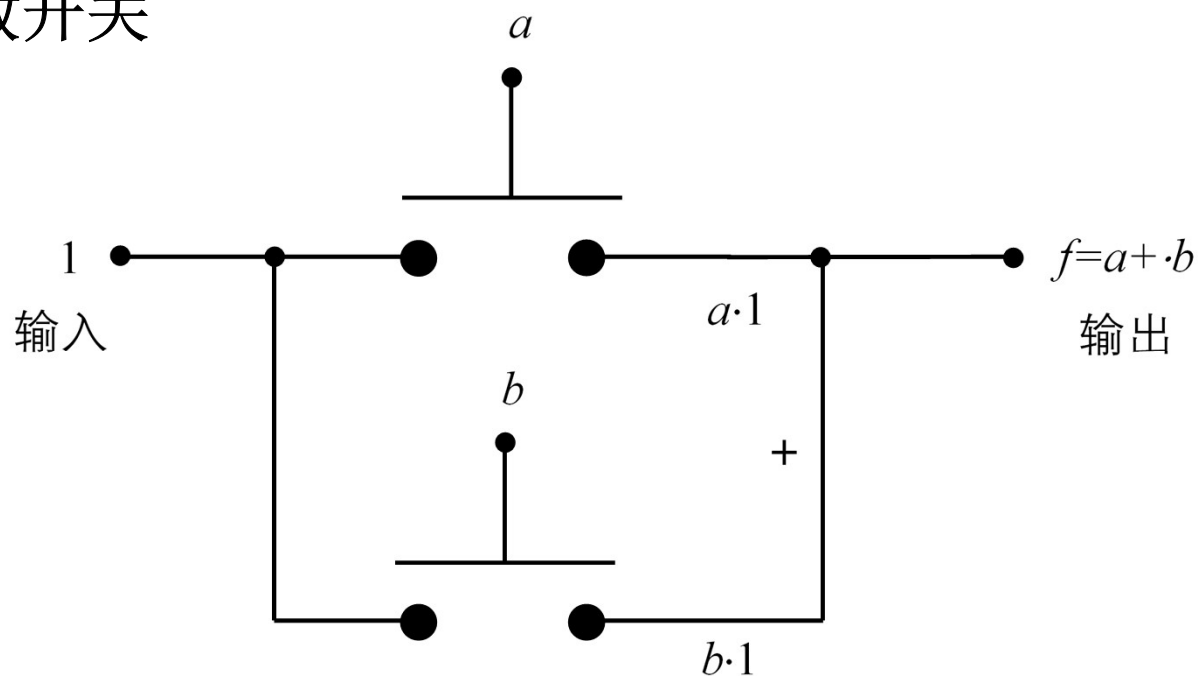
高电平有效开关串联

$$g = (a \cdot 1) \cdot b = a \cdot b$$

逻辑与 (AND)

布尔运算与理想开关

高电平有效开关

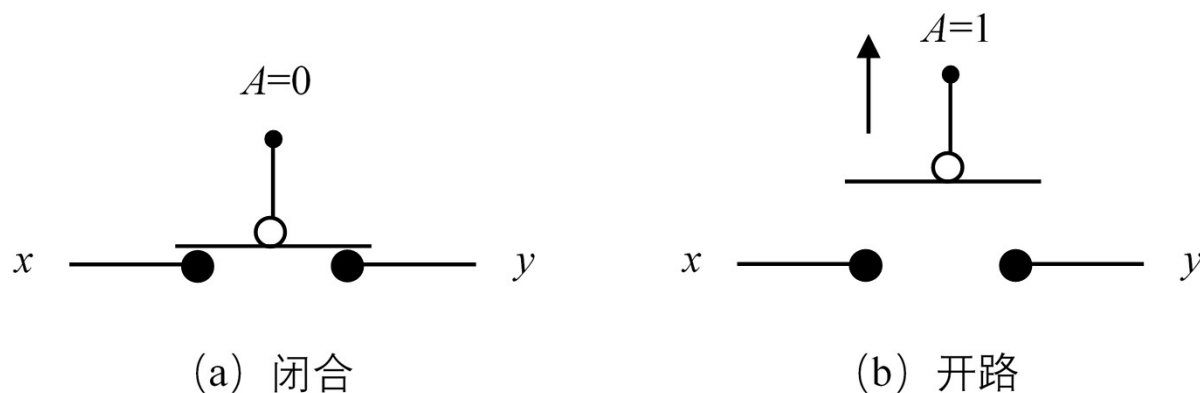


高电平有效开关并联

逻辑或 (OR)

布尔运算与理想开关

低电平有效开关

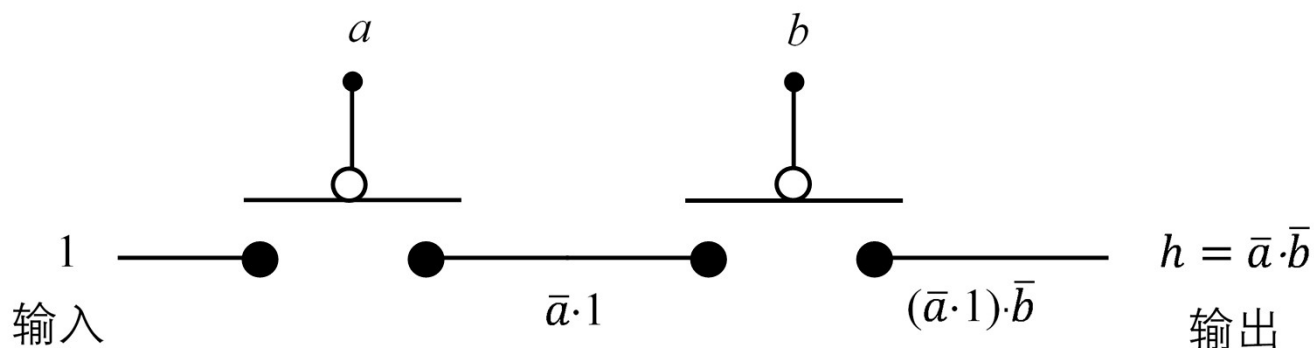


$$y = x \cdot \bar{A} \quad \text{iff } A = 0$$

当 $A=1$ 时， x 与 y 之间的关系是不确定的

布尔运算与理想开关

低电平有效开关



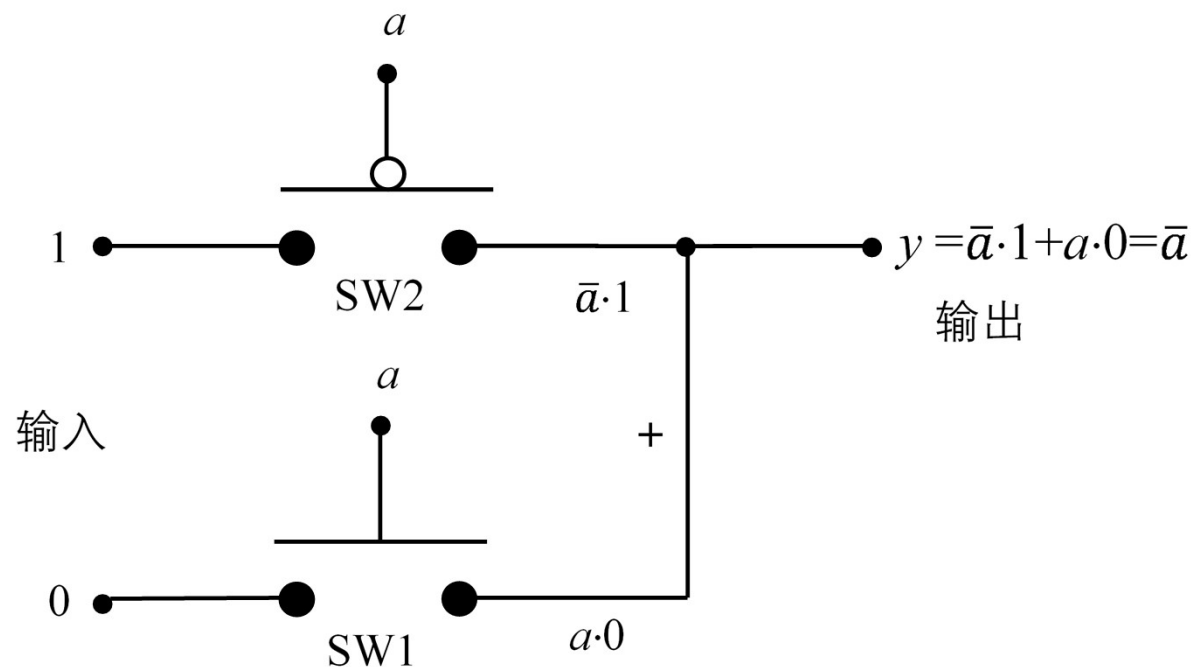
低电平有效开关串联

$$h = (\bar{a} \cdot 1) \cdot \bar{b} = \overline{a + b}$$

或非 (NOR)

布尔运算与理想开关

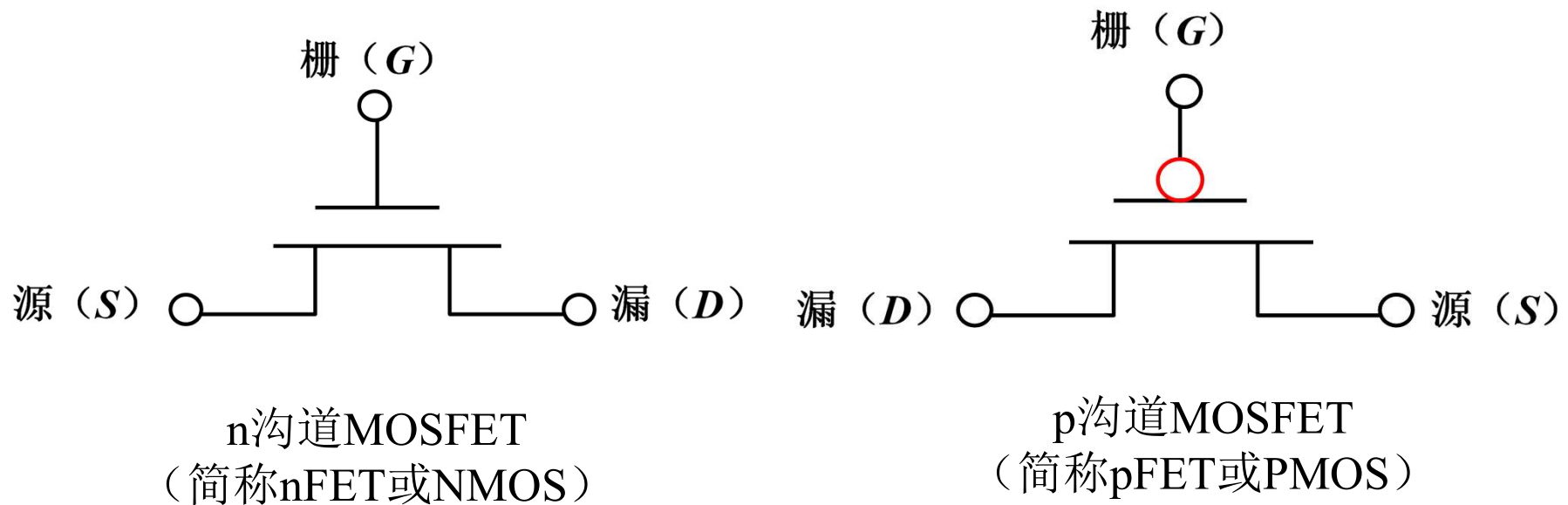
基于开关的非门



$$y = \bar{a} \cdot 1 + a \cdot 0 = \bar{a}$$

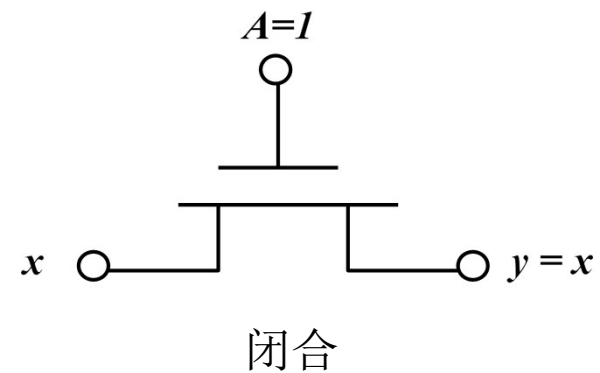
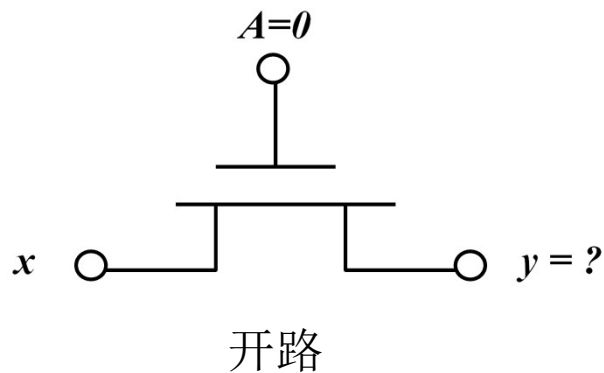
MOSFET开关

MOSFET (Metal-Oxide-Semiconductor Field-Effect Transistor) ,
金属氧化物半导体场效应晶体管



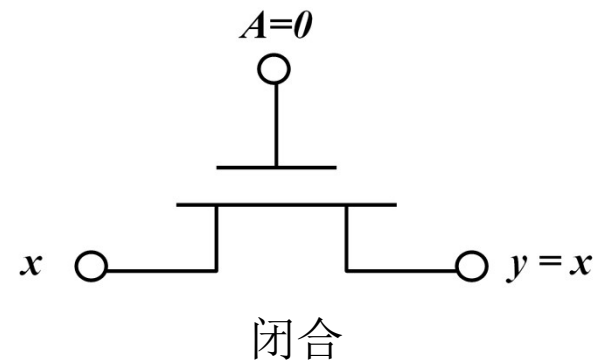
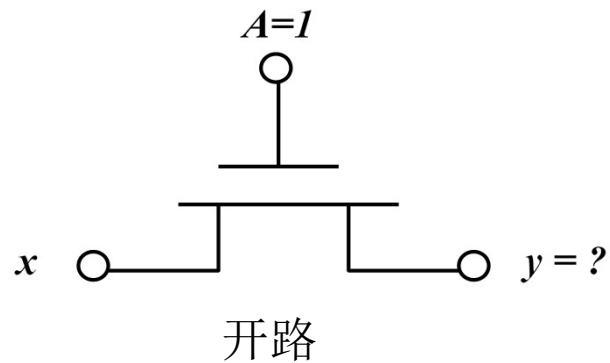
MOSFET开关

NMOS开关特性



$$y = x \cdot A$$

PMOS开关特性

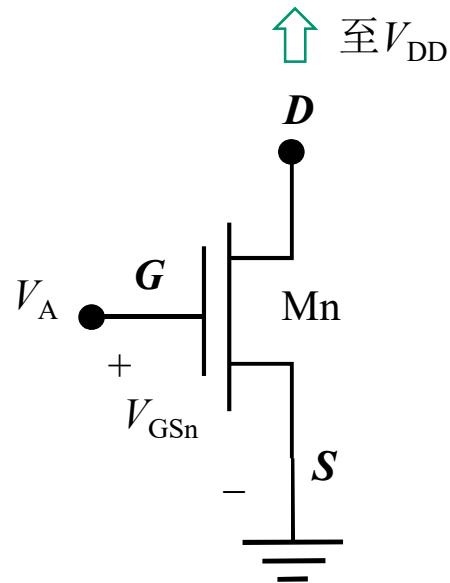


$$y = x \cdot \bar{A}$$

MOSFET开关

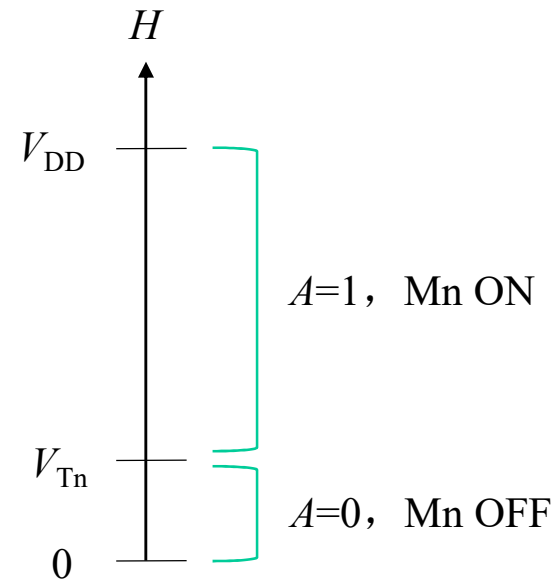
MOSFET阈值电压

NMOS



栅-源电压

$$V_A = V_{GSn}$$



逻辑含义

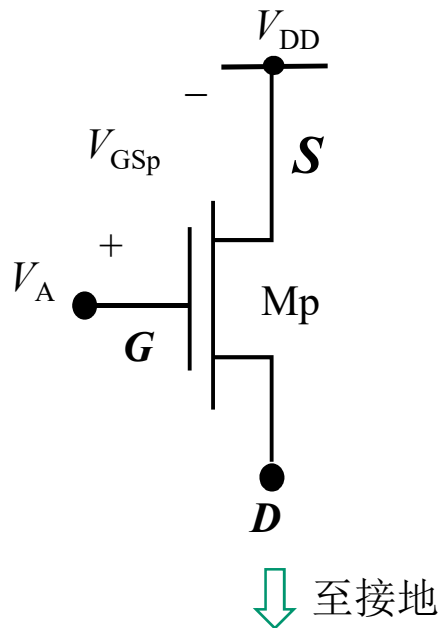
$$V_A \leq V_{Tn}, A = 0$$

$$V_A \geq V_{Tn}, A = 1$$

MOSFET开关

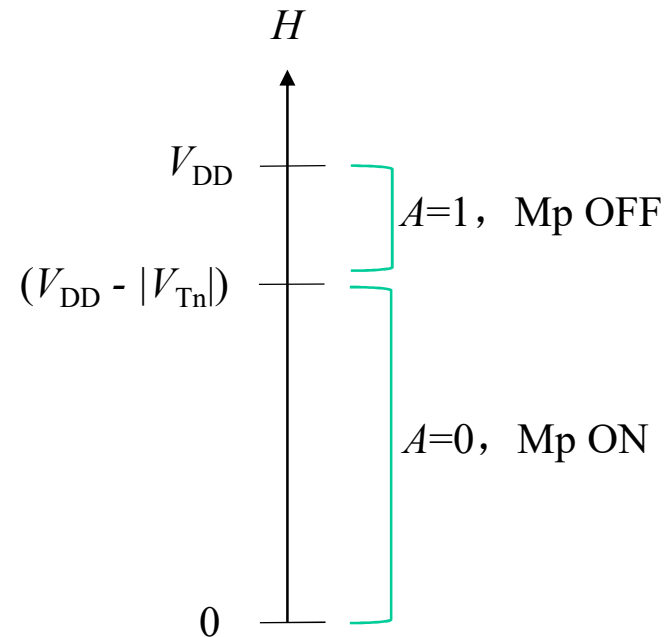
MOSFET阈值电压

PMOS



栅-源电压

$$V_A = V_{GSsp} + V_{DD} = V_{DD} - |V_{GSsp}|$$



逻辑含义

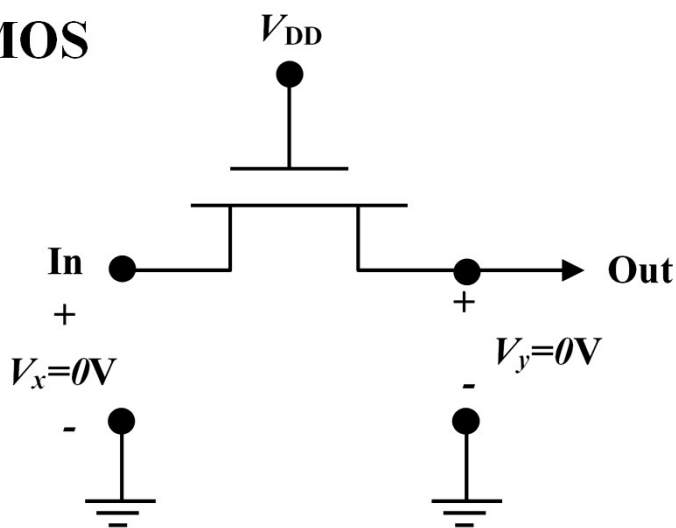
$$V_A \leq V_{DD} - |V_{Tp}|, A=0$$

$$V_A \geq V_{DD} - |V_{Tp}|, A=1$$

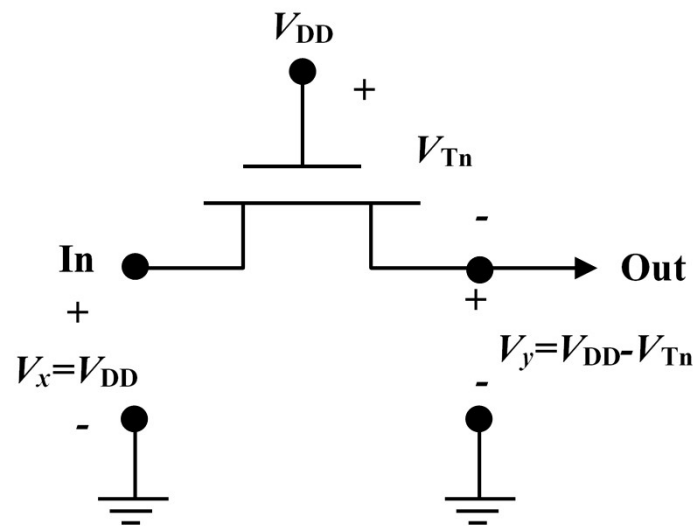
MOSFET开关

MOSFET传输特性

NMOS



传输逻辑0



传输逻辑1

NMOS可以传输的电压范围： $[0, V_{DD} - V_{Tn}]$

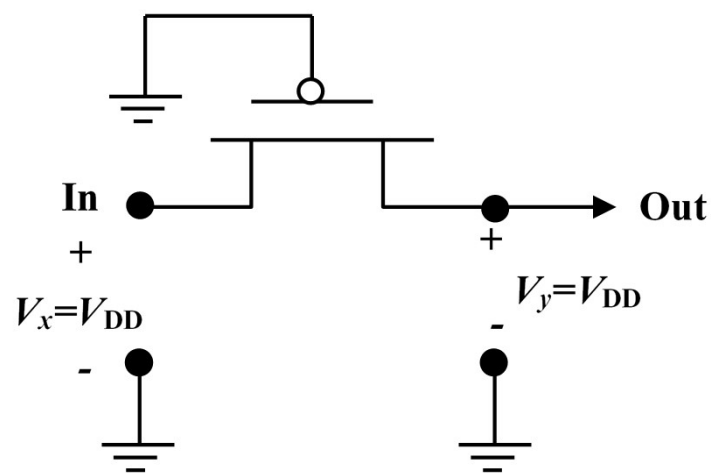
传输高电平时，小于理想的逻辑1电压 V_{DD} ，称其传输一个“弱”逻辑1

传输低电平时，能产生一个输出电压 $0V$ ，称它传输一个“强”逻辑0

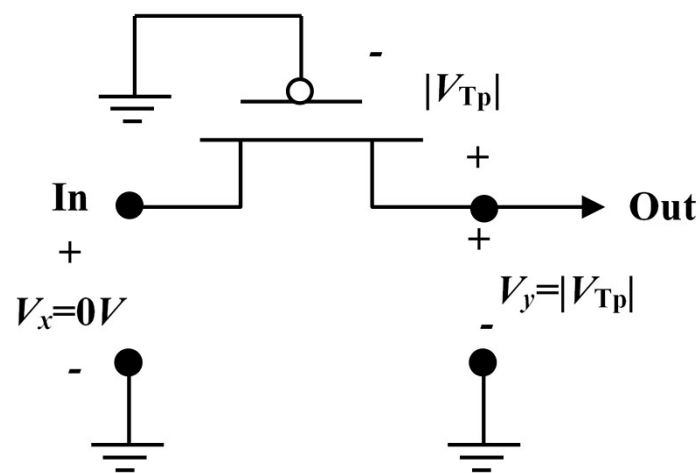
MOSFET开关

MOSFET传输特性

PMOS



传输逻辑1



传输逻辑0

PMOS可以传输的电压范围： $[V_{DD}, |V_{Tp}|]$

传输高电平时，能输出理想的逻辑1电压 V_{DD} ，传输“强”逻辑1

传输低电平时，只能产生最低输出电压 $|V_{Tp}|$ ，传输“弱”逻辑0

MOSFET开关

MOSFET传输特性

总结:

- NMOS传输强逻辑0电压，但传输弱逻辑1电平
- PMOS传输强逻辑1电压，但传输弱逻辑0电平

设计互补MOS（CMOS）电路解决传输电平的问题:

- 使用PMOS传输逻辑1电压 V_{DD}
- 使用NMOS传输逻辑0电压（GND, 0V）

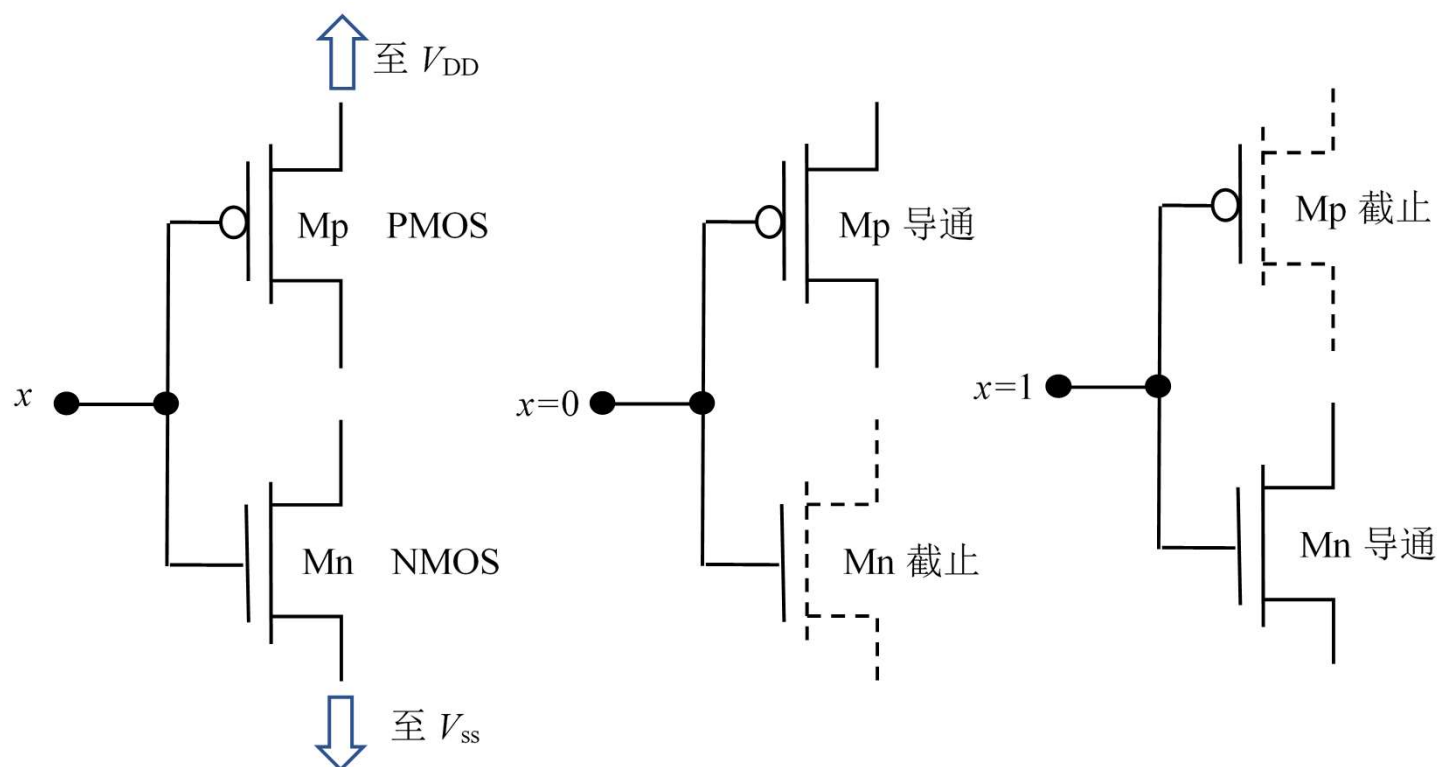
因此能够构建一个可传输理想逻辑电压0V和 V_{DD} 的电路

布尔运算与CMOS逻辑门

- 二进制数和二进制逻辑
- 布尔函数
- 范式与标准式
- MOSFET开关
- 基本的CMOS逻辑门
- 门电路化简
- CMOS复合逻辑门

基本的CMOS逻辑门

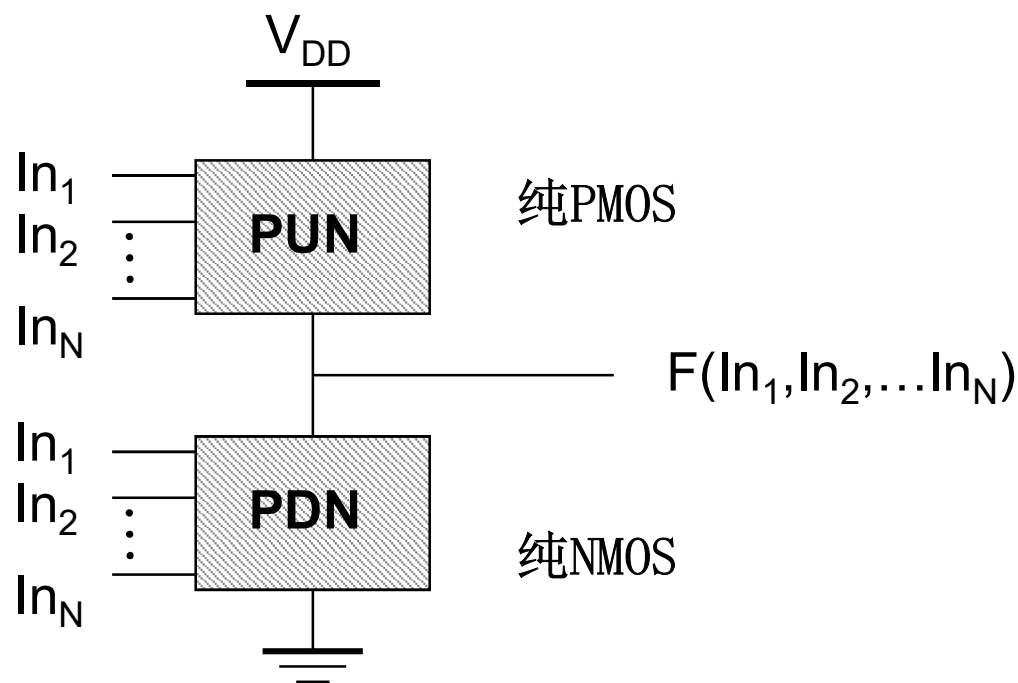
CMOS互补对



NMOS与PMOS的电气特性正好相反，是一种耦合的开关方式

基本的CMOS逻辑门

静态CMOS逻辑门由PUN（上拉网络）和PDN（下拉网络）组合而成



PUN 和 PDN 是对偶逻辑网络，在稳定状态时有且仅有一个网络导通
一旦瞬态过程完成，总有一条路径存在于电源 V_{DD} 和输出端 F 之间（输出高电平逻辑1）或存在于地和输出端 F 之间（输出低电平逻辑0）

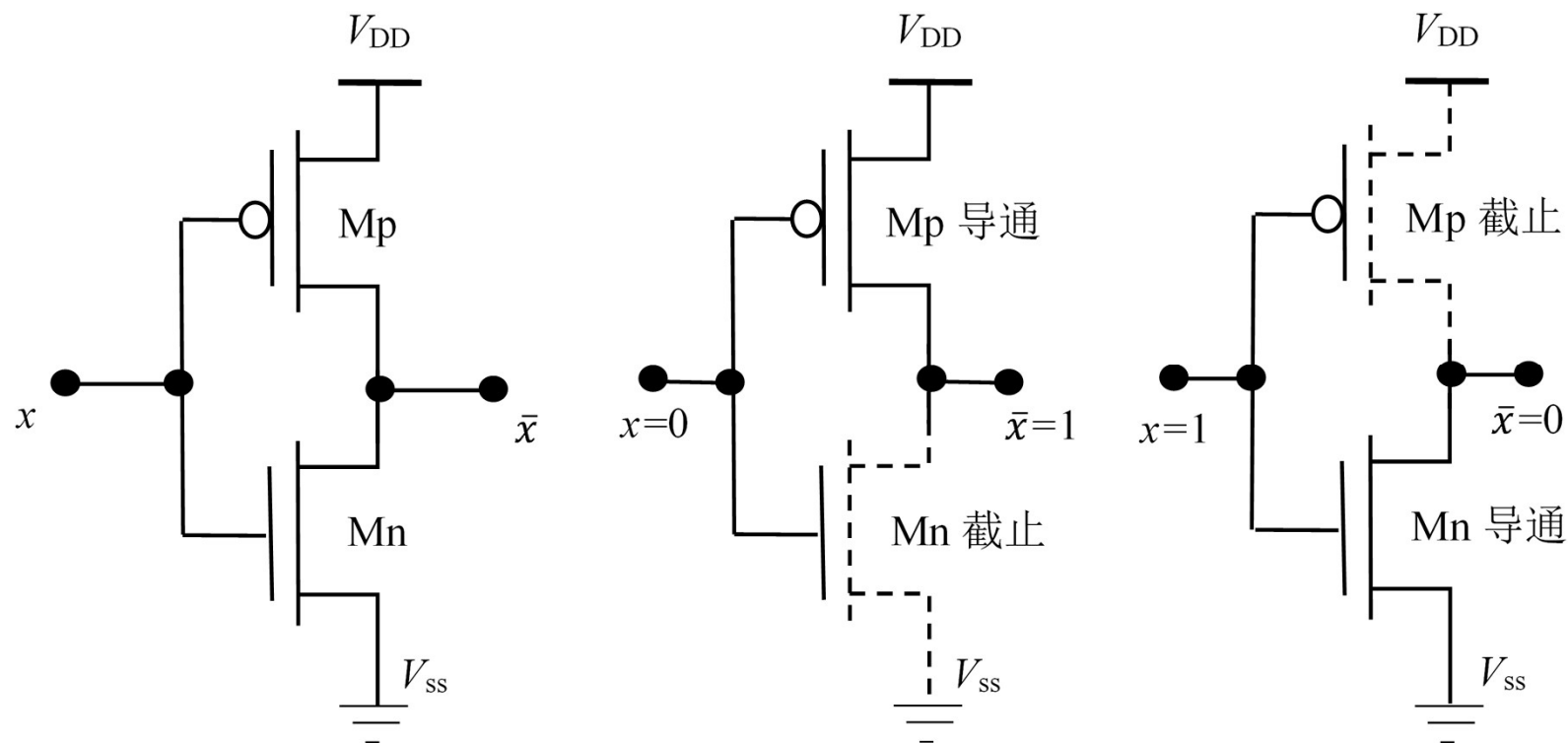
基本的CMOS逻辑门

PUN和PDN构造的经验规则：

- 晶体管看作是由其栅端信号控制的开关
- **PDN**用**NMOS**器件，**PUN**用**PMOS**器件
- 推导构造逻辑功能的规则：**NMOS**串联对应与（**AND**）功能，**NMOS**并联对应或（**OR**）功能
- 根据**Morgan**定理，**PUN**和**PDN**是对偶网络。构造**CMOS**门，一个网络（如**PDN**）用串并联器件的组合实现，另一个网络可根据对偶性原理得到
- 互补门自然求反功能。单级只能实现**NAND**，**NOR**和**XNOR**等，非反相布尔函数需要额外的反相器
- 实现**N**输入的逻辑门需要晶体管数目为**2N**

基本的CMOS逻辑门

CMOS反相器 (INV)

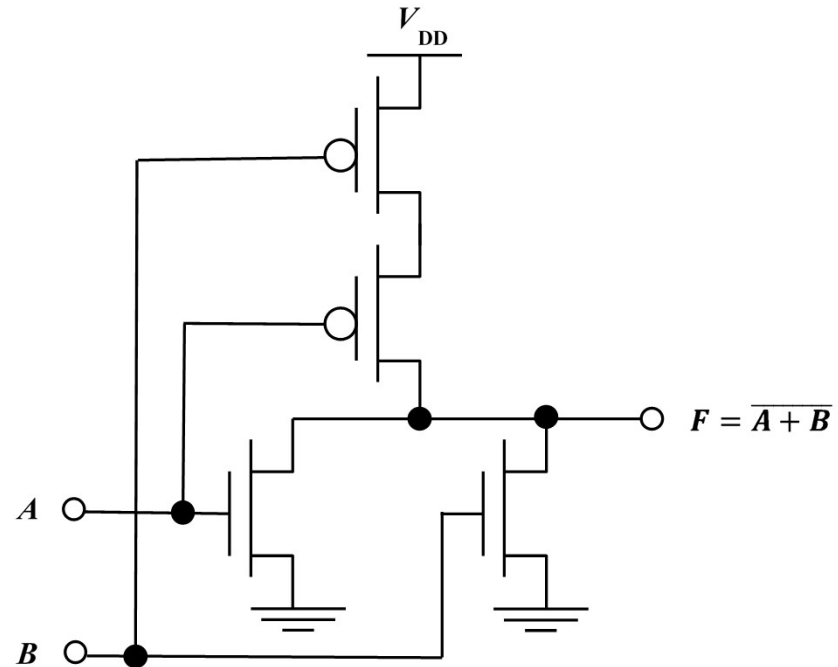


一个MOSFET互补对

基本的CMOS逻辑门

CMOS或非（NOR）门

- 对每个输入使用一个NMOS/PMOS互补对
- 将输出节点通过PMOS组成的PUN与电源VDD相连，通过NMOS组成的PDN与地相连
- 根据NMOS和PMOS串、并联规则确定PUN和PDN内部连接关系



二输入或非门NOR2

$$F(A, B) = \overline{A + B}$$

PUN（低电平有效的开关串联）： $h = \overline{A} \cdot \overline{B} = \overline{A + B}$

PDN（高电平有效的开关并联）： $g = A + B$

$$F(A, B) = h \cdot 1 + g \cdot 0 = \overline{A + B}$$

基本的CMOS逻辑门

CMOS与非（NAND）门

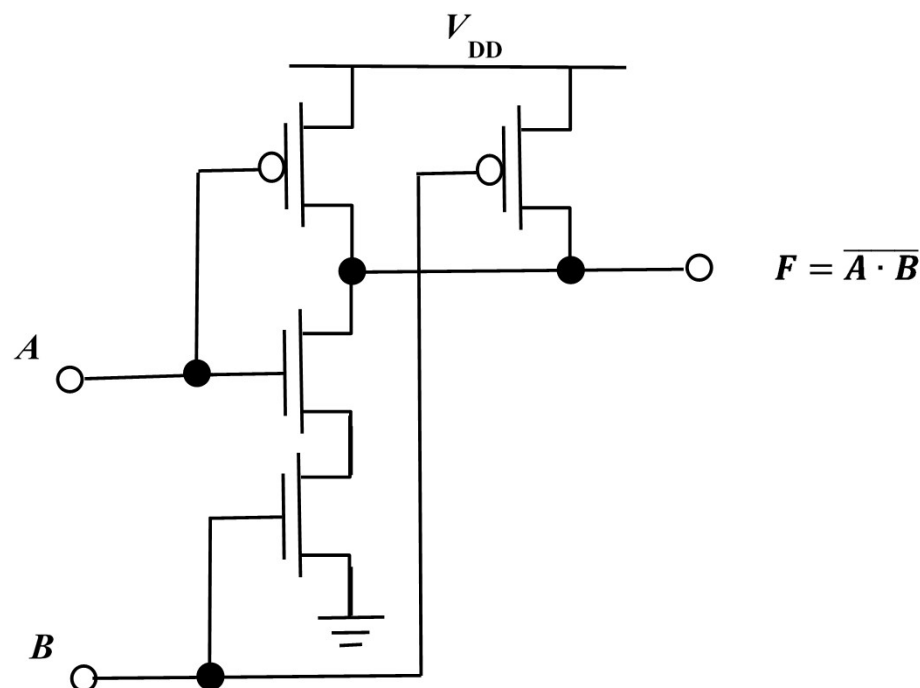
二输入与非门NAND2

$$F(A, B) = \overline{A \cdot B}$$

$$\text{PUN: } h = \bar{A} + \bar{B} = \overline{A \cdot B}$$

$$\text{PDN: } g = A \cdot B$$

$$F(A, B) = h \cdot 1 + g \cdot 0 = \overline{A \cdot B}$$



可以验证输出 F 总是连接到 V_{DD} 或地，但绝不会同时连接到二者

基本的CMOS逻辑门

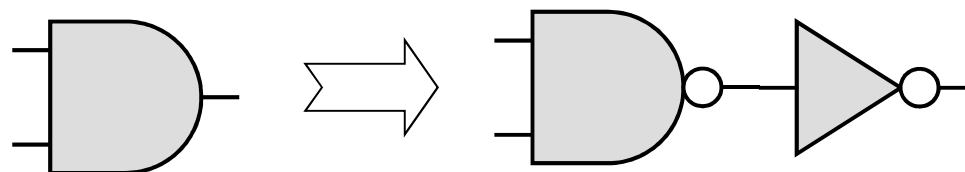
互补CMOS逻辑风格

PUN与PDN是对偶关系

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

互补逻辑门是反相输出



$$\text{AND} = \text{NAND} + \text{INV}$$

布尔运算与CMOS逻辑门

- 二进制数和二进制逻辑
- 布尔函数
- 范式与标准式
- MOSFET开关
- 基本的CMOS逻辑门
- 门电路化简
- CMOS复合逻辑门

门级化简

- 找到描述数字电路的布尔函数的最佳门级实现
- 图形法化简

图形法化简

■ 代数方法化简

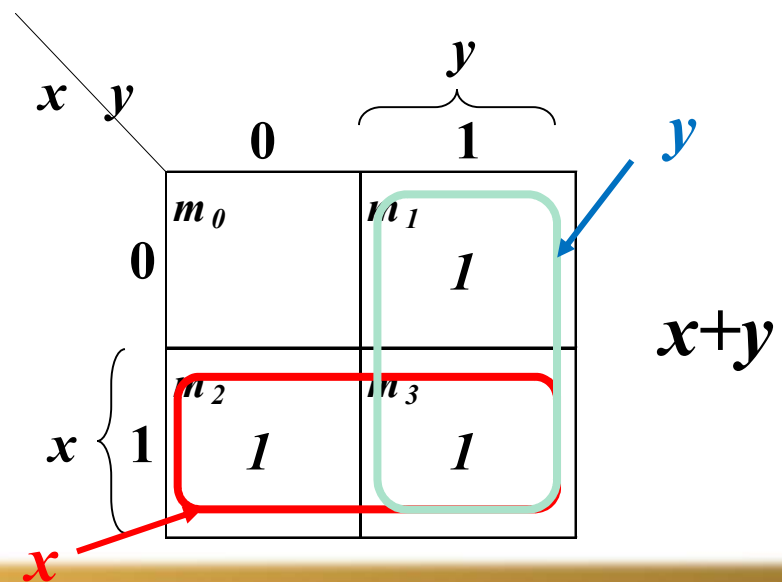
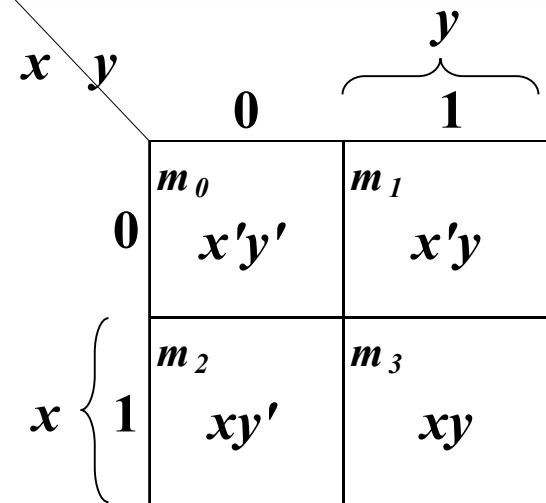
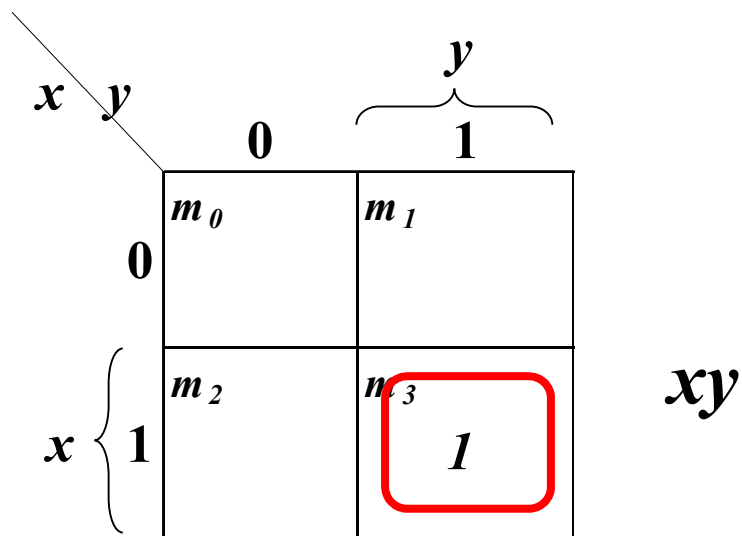
- 简化步骤没有具体规则

■ 图形法化简

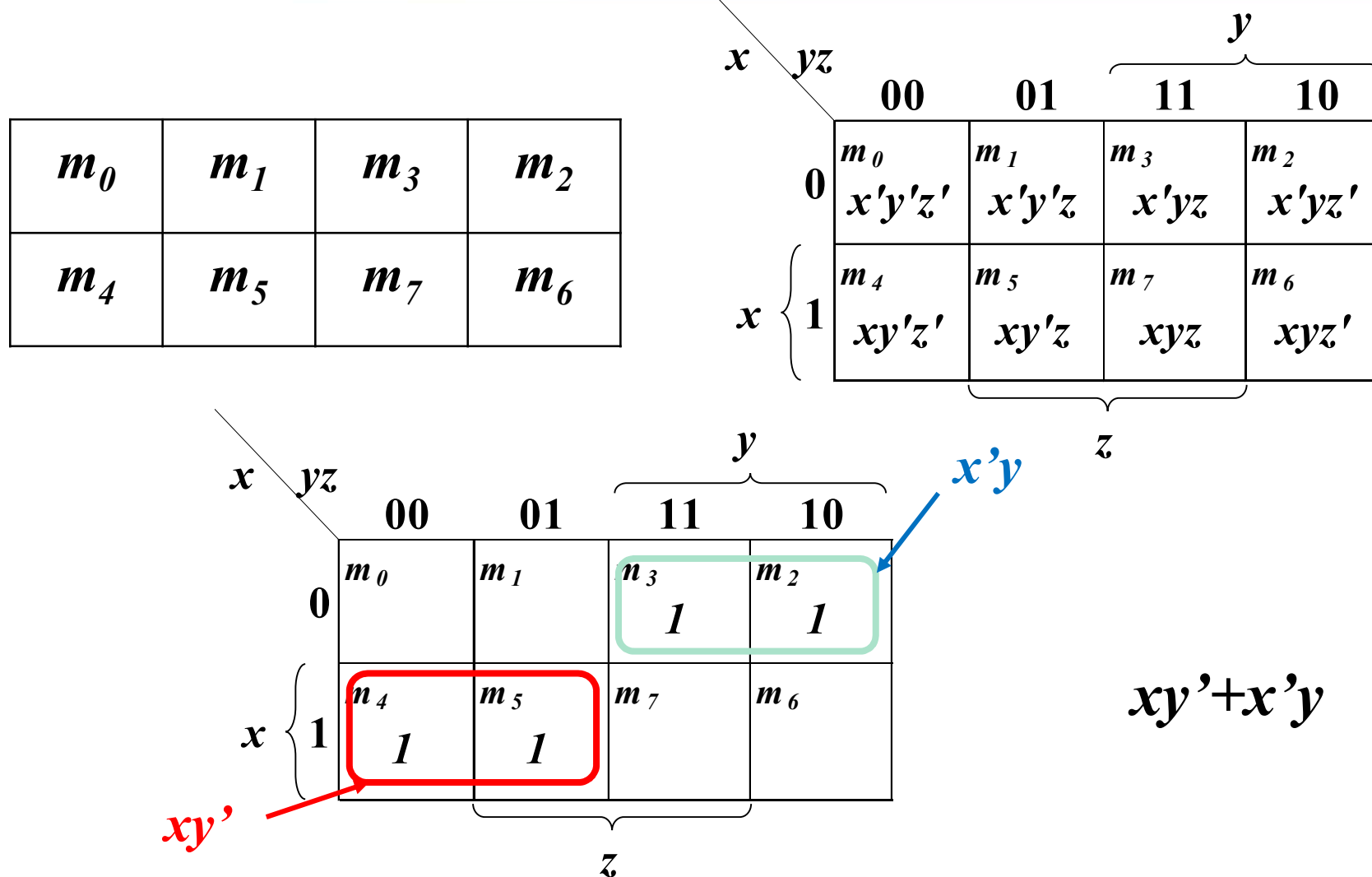
- 简单、直观
- 真值表的图形化表示
- 卡诺图或K图
 - ◆ 由一个个方格组成，每个方格代表一个最小项
 - ◆ 图中含有布尔函数最小项的方格圈在一起，根据画圈的图形表示布尔函数
 - ◆ 得到的最简式是两种标准表达式之一：积之和式、和之积式
 - ◆ 最简表达式并不是唯一的，可能有两种或更多

两变量卡诺图

m_0	m_1
m_2	m_3

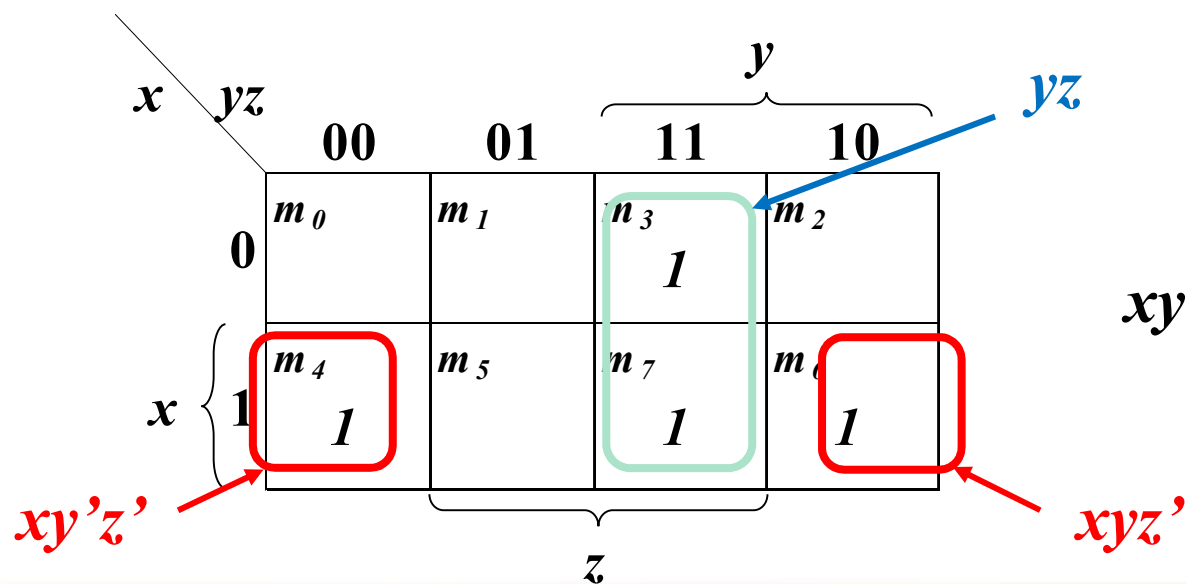


三变量卡诺图



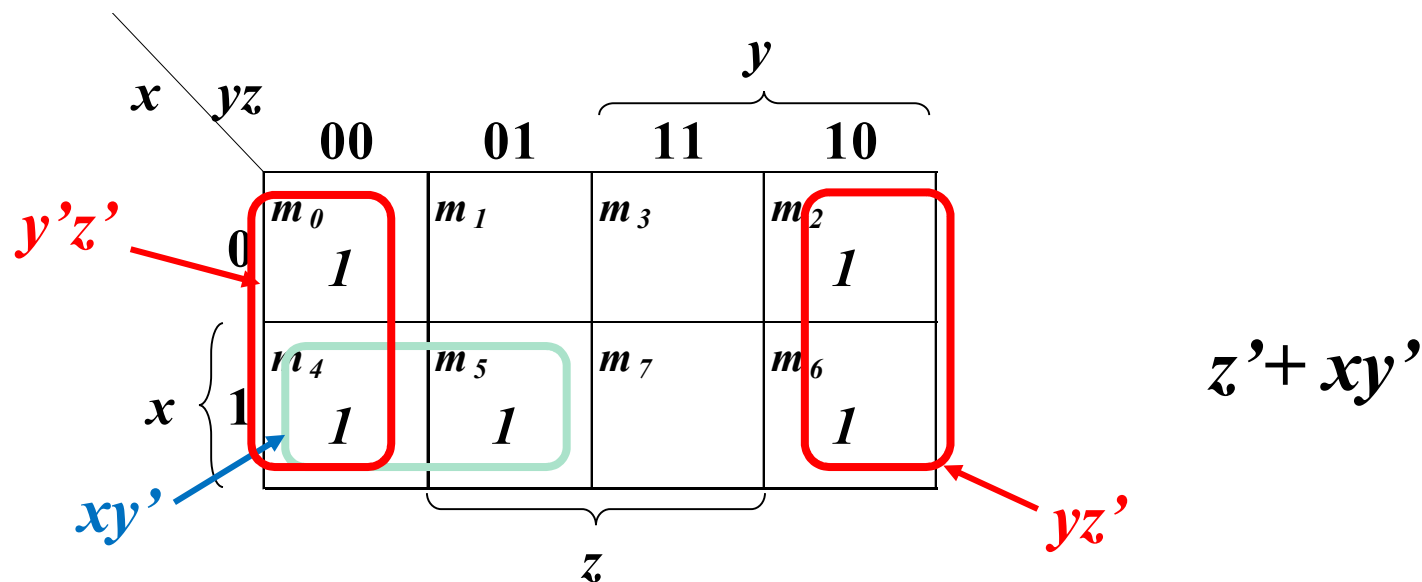
三变量卡诺图

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6



三变量卡诺图

三变量卡诺图中，任意四个相邻方格合并

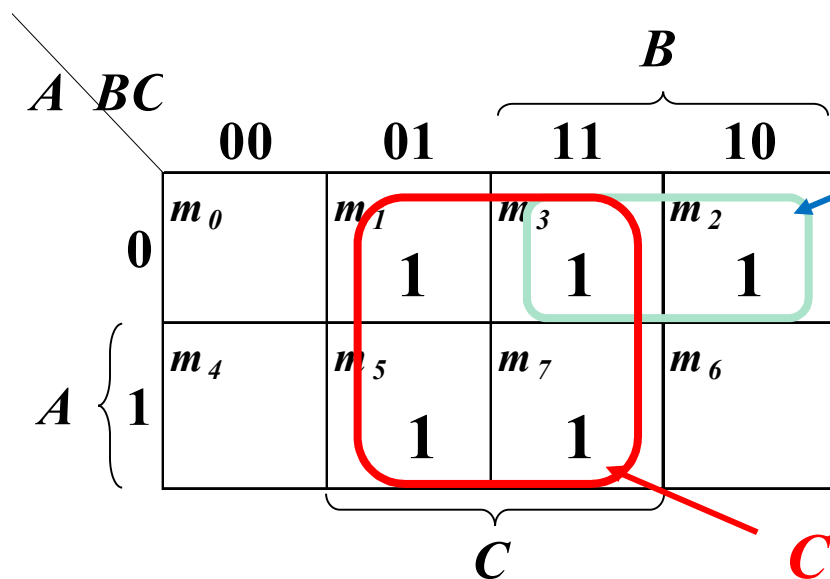


可以合并的相邻小方格数是2的幂次（如1、2、4、8）。
被合并的相邻方格数目越大，得到的乘积项中包含的字母变量越少。

三变量卡诺图

布尔函数: $F = A'C + A'B + AB'C + BC$

- 1) 将其表示成最小项之和
- 2) 求最小的乘积项之和



$$\begin{aligned} F(A, B, C) &= \sum (1, 2, 3, 5, 7) \\ &= C + A'B \end{aligned}$$

四变量卡诺图

■ 四变的量卡诺图

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

$wx \ yz$		y			
		00	01	11	10
w	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
	01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
	10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$
		z			

x

四变量卡诺图

■ 四变量卡诺图的化简方法与三变量的是相类似的：

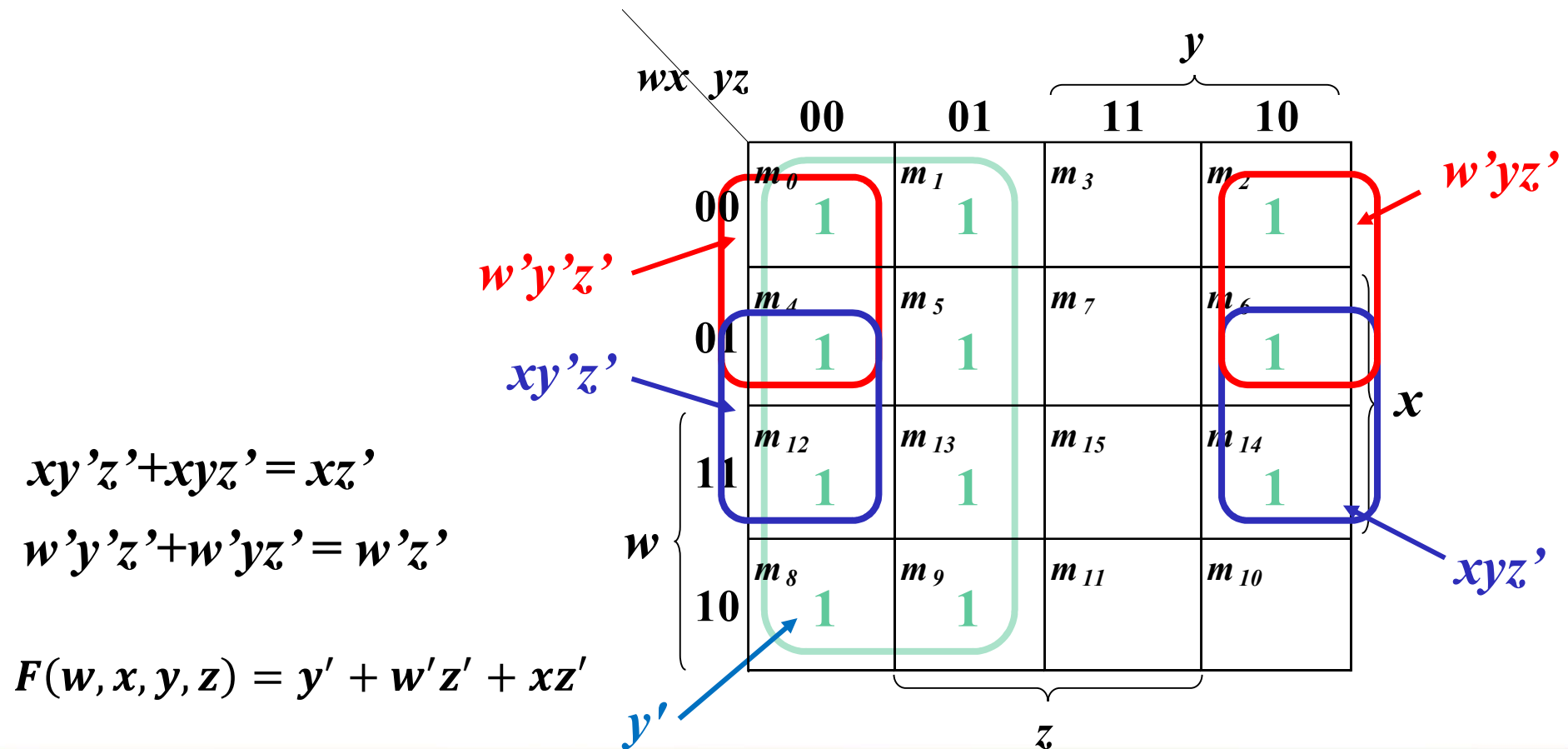
- 紧挨着的方格是相邻的方格
- 同一列的最顶端和最低端、同一行的最左边和最右边的方格也是相邻的

$wx \backslash yz$		y			
		00	01	11	10
w	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
	01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
	10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$

z

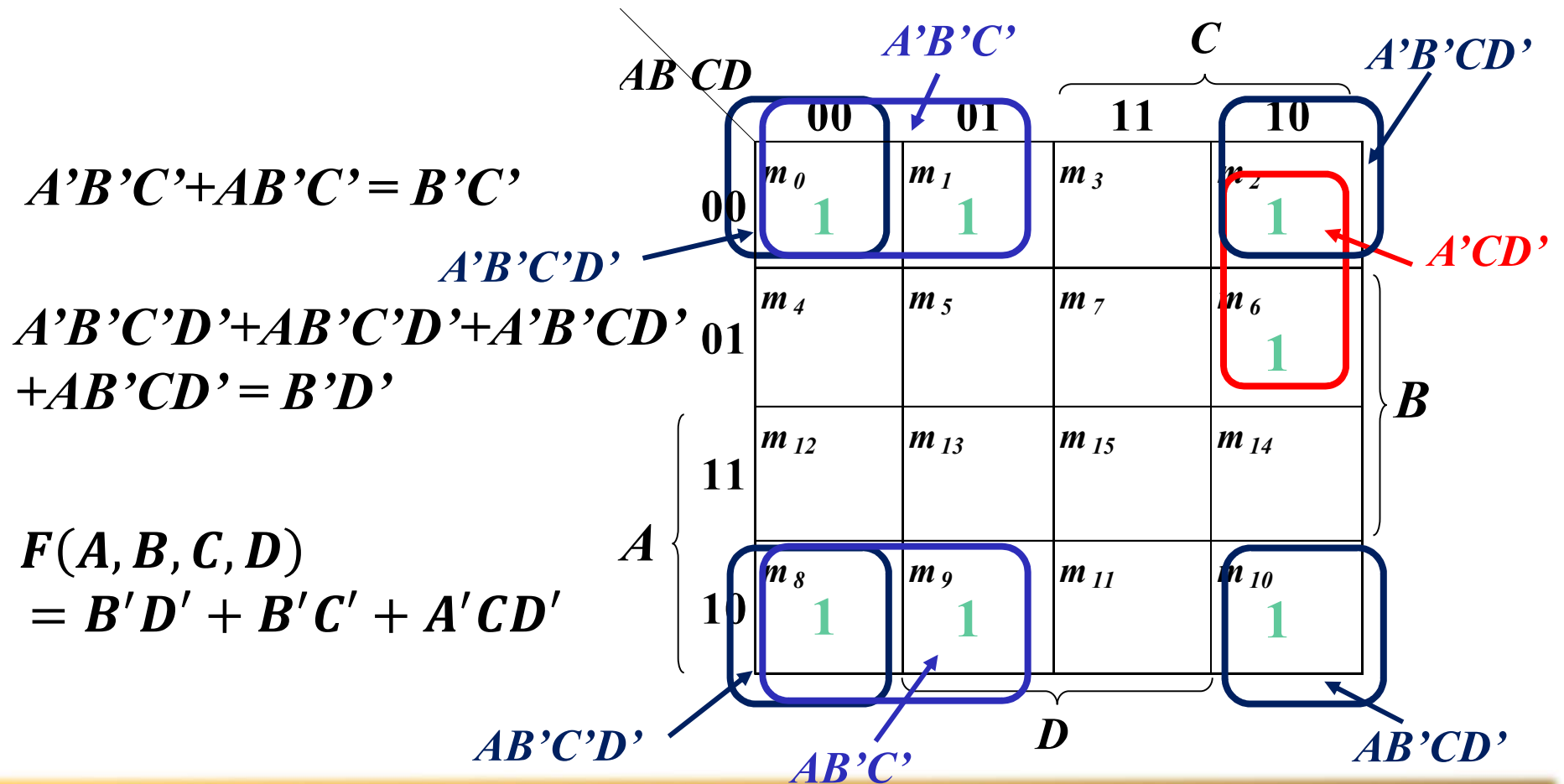
四变量卡诺图

■ 化简布尔函数: $F(w, x, y, z) = \sum (0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$



四变量卡诺图

■ 化简布尔函数: $F = A'B'C' + B'CD' + A'BCD' + AB'C'$



四变量卡诺图

- 在卡诺图中选择相邻方格时，必须确保：
 - 1) 合并的方格能覆盖函数中所有的最小项
 - 2) 表达式中的项数最少
 - 3) 没有冗余项（那些已经被其他项涵盖的最小项）有时候可能有两个或更多个表达式满足最简原则
- 对卡诺图的化简过程变得更加系统化，可使用**质蕴含（prime implicant）**的概念
 - 质蕴含是指合并卡诺图中最大可能数目的相邻方格而得到的乘积项
 - 如果一个最小项对应的方格只被一个质蕴含所包含，称之为**基本质蕴含**

四变量卡诺图

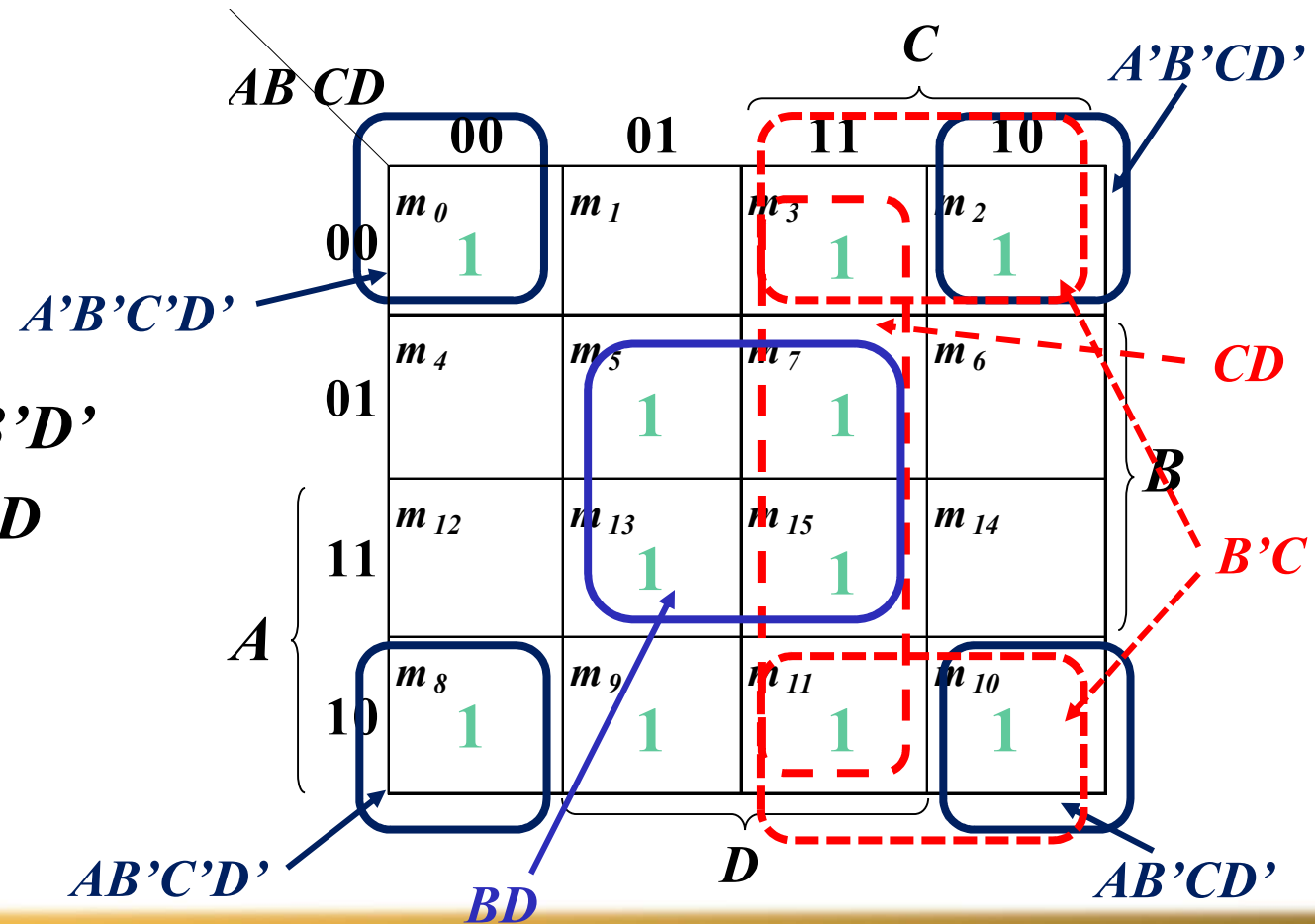
- 质蕴含

$$F(A, B, C, D) = \sum (0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

$m_0, m_2, m_8, m_{10}: B'D'$

$m_5, m_7, m_{13}, m_{15}: BD$

$m_3: CD \text{ 或 } B'C$



四变量卡诺图

- 质蕴含

$$F(A, B, C, D) = \sum (0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

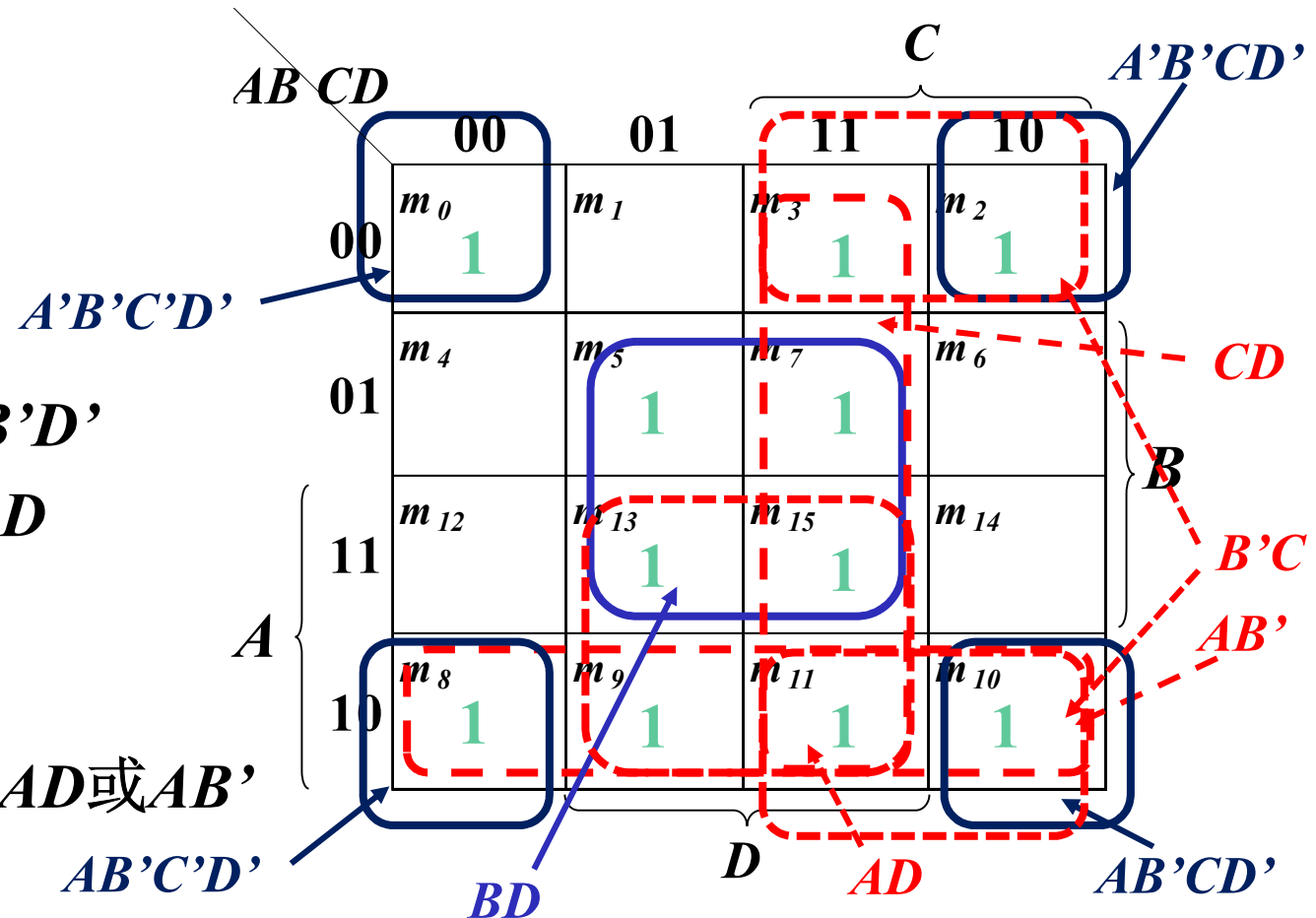
$m_0, m_2, m_8, m_{10}: B'D'$

$m_5, m_7, m_{13}, m_{15}: BD$

$m_3: CD \text{ 或 } B'C$

$m_9: AD \text{ 或 } AB'$

$m_{11}: CD, B'C, AD \text{ 或 } AB'$



四变量卡诺图

• 质蕴含

$$F(A, B, C, D) = \sum (0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

$$\begin{aligned} F &= BD + B'D' + CD + AD \\ &= BD + B'D' + CD + AB' \\ &= BD + B'D' + B'C + AD \\ &= BD + B'D' + B'C + AB' \end{aligned}$$

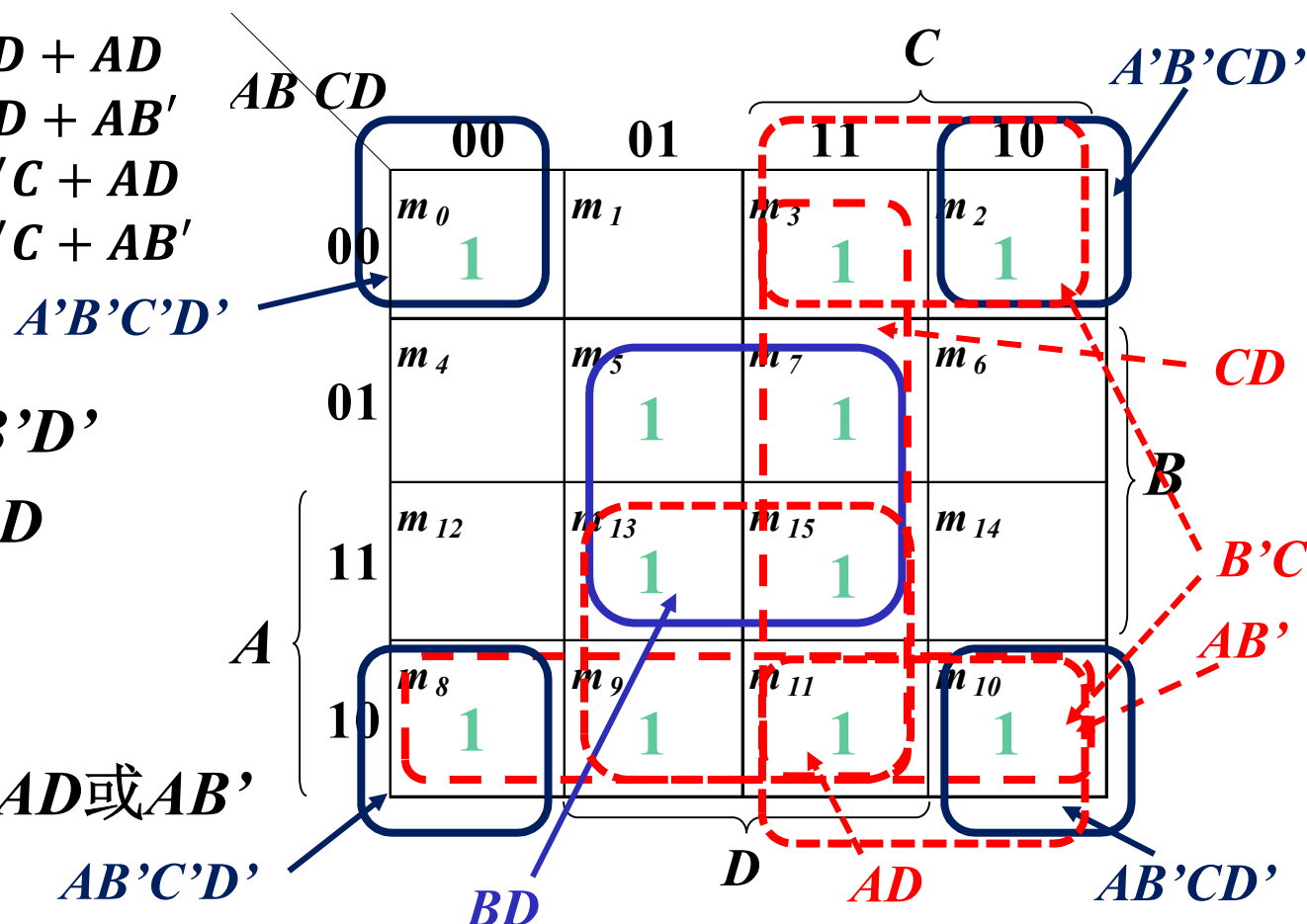
$$m_0, m_2, m_8, m_{10}: B'D'$$

$$m_5, m_7, m_{13}, m_{15}: BD$$

$$m_3: CD \text{ 或 } B'C$$

$$m_9: AD \text{ 或 } AB'$$

$$m_{11}: CD, B'C, AD \text{ 或 } AB'$$



四变量卡诺图

- 在卡诺图中质蕴含的识别能够帮助确定最简代数式
 - 从卡诺图中得到最简代数式的过程，要求首先确定所有的基本质蕴含
 - 所有基本质蕴含之和加上其他质蕴含（覆盖不能被基本质蕴含圈过的最小项），就得到了最简表达式
 - 有时候可能有多种方法来合并相邻方格，从而得到等价的最简表达式

五变量卡诺图

- 超过四个变量的卡诺图使用起来比较复杂
- 五个变量的卡诺图有**32**个方格、六个变量的卡诺图有**64**个方格，...

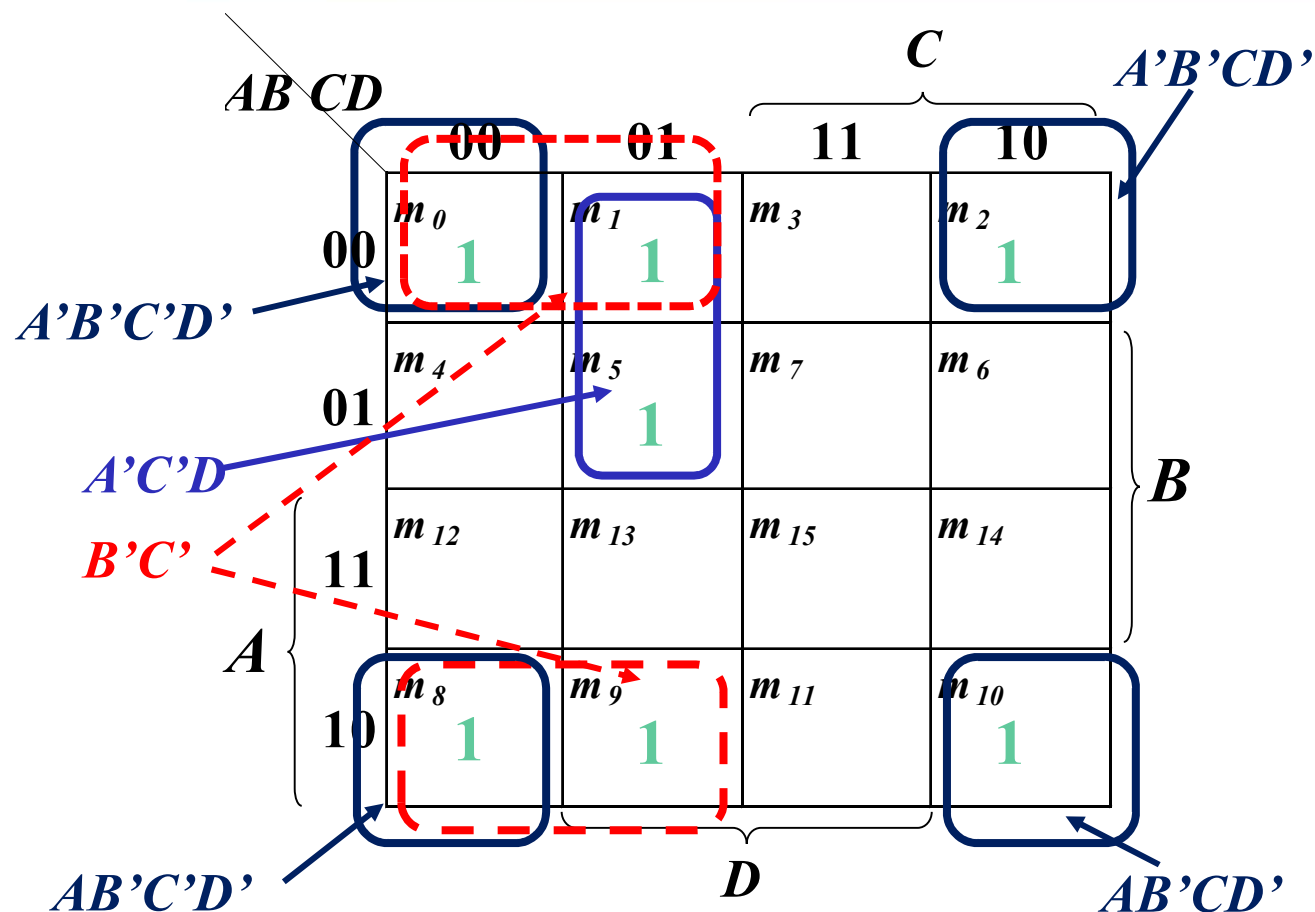
和之积式的化简

■ 从卡诺图得到和之积式表达式:

- 卡诺图中标**1**的方格代表函数的最小项
- 没有被函数包含的最小项属于该函数的反函数，也就是说，反函数可以用卡诺图中没有标**1**的方格表示
- 将空的（没有标**1**的）方格标上**0**，并对相邻方格进行合并，得到反函数的积之和最简表达式（即**F'**）
- 对**F'**取反得到原函数**F**的和之积式（摩根定理）

例：化简布尔函数 $F(A, B, C, D) = \sum(0, 1, 2, 5, 8, 9, 10)$

和之积式的化简



合并标1的方格，得到积之和式： $F = B'D' + B'C' + A'C'D$

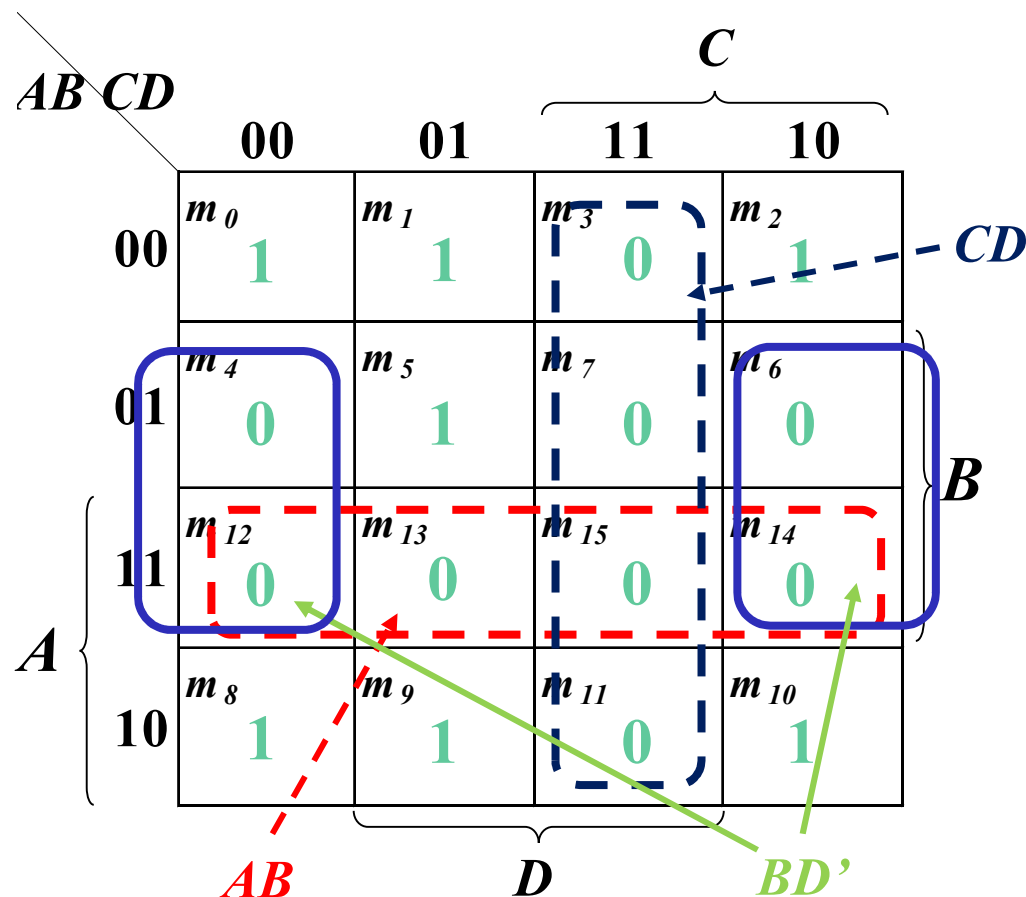
和之积式的化简

- 合并标0的方格，得到最简反函数表达式：

$$F' = AB + CD + BD'$$

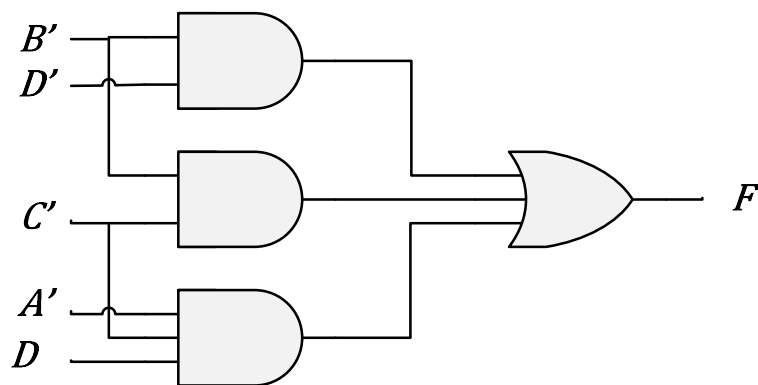
- 应用摩根定理，得到函数和之积式最简表达式：

$$\begin{aligned} F &= (AB + CD + BD')' \\ &= (A' + B')(C' + D')(B' + D) \end{aligned}$$



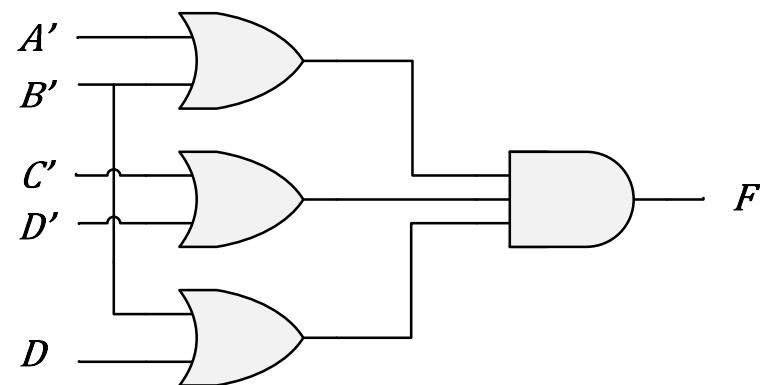
和之积式的化简

■ 电路图



$$F = B'D + B'C' + A'C'D$$

与-或



$$F = (A' + B')(C' + D')(B' + D)$$

或-与

无关条件

- 有的应用场合，函数对某个变量组合可以是不确定的
 - 例如，用四位二进制表示十进制数时，有六项没有用到，因而称为不确定项
 - 在大多数情况下，不需要关心这些不确定项到底取什么值。因此，习惯上把函数的不确定最小项称为“**无关条件**”，或**任意条件**、**任意项**
- 无关条件可以用于对布尔表达式做进一步简化
 - 无关条件最小项是逻辑值不确定的变量组合，这样的最小项在卡诺图中不标**1**，也不标**0**，而是**用X来表示**
 - 当在卡诺图中相邻方格合并化简时，可以将**标X的方格任意假设为0或者1**，从而得到更加简化的表达式

无关条件

■ 例：化简布尔函数

$$F(w, x, y, z) = \sum (1, 3, 7, 11, 15)$$

无关条件（任意项）为：
 $d(w, x, y, z) = \sum (0, 2, 5)$

		y			
		00	01	11	10
w	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}

无关条件

化简布尔函数

$$F(w, x, y, z) = \sum (1, 3, 7, 11, 15)$$

无关条件（任意项）为：
 $d(w, x, y, z) = \sum (0, 2, 5)$

$$F = yz + w'z$$

或

$$F = yz + w'x'$$

		y			
		yz		11	10
w	wx	00	01	11	10
	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
w	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0

无关条件

$$F(w, x, y, z) = yz + w'z = \sum (1, 3, \mathbf{5}, 7, 11, 15)$$

$$F(w, x, y, z) = yz + w'x' = \sum (\mathbf{0}, 1, \mathbf{2}, 3, 7, 11, 15)$$

		y			
		yz		11	10
w	wx	00	01	11	10
	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0

Diagram illustrating a 4x4 Karnaugh map for the function $F(w, x, y, z) = yz + w'z$. The map is labeled with w (rows) and x (columns). The variables y and z are indicated by brackets above and below the columns. The map shows the function's value (0, 1, or X) for each combination of w and x . The function is 1 for the following minterms: $m_1, m_3, m_5, m_7, m_{11}, m_{15}$. The function is 0 for the following minterms: $m_4, m_6, m_{12}, m_{14}, m_8, m_9, m_{10}$. The function is X (don't care) for the following minterms: m_0, m_2, m_5 . The map is divided into four groups by dashed lines: a group of 1s (minterms 1, 3, 5, 7) and a group of 1s (minterms 11, 15). The group of 1s (minterms 1, 3, 5, 7) is highlighted with a blue dashed line. The group of 1s (minterms 11, 15) is highlighted with a red solid line.

无关条件

$$F' = z' + wy'$$

对 F' 取反

$$F(w, x, y, z) = z(w' + y)$$

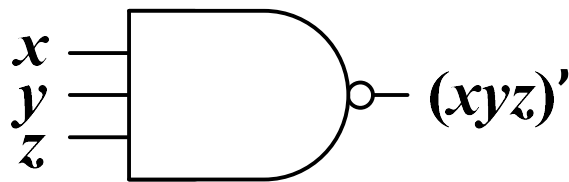
$$= \sum (1, 3, \mathbf{5}, 7, 11, 15)$$

		y			
		yz		11	10
w	x	00	01	11	10
	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
w	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0

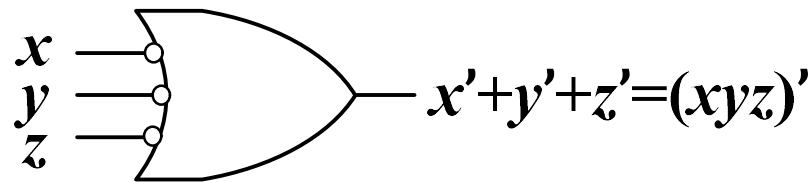
NAND和NOR实现逻辑函数

■ NAND实现布尔函数

- 两种等价的与非门表示



与-非

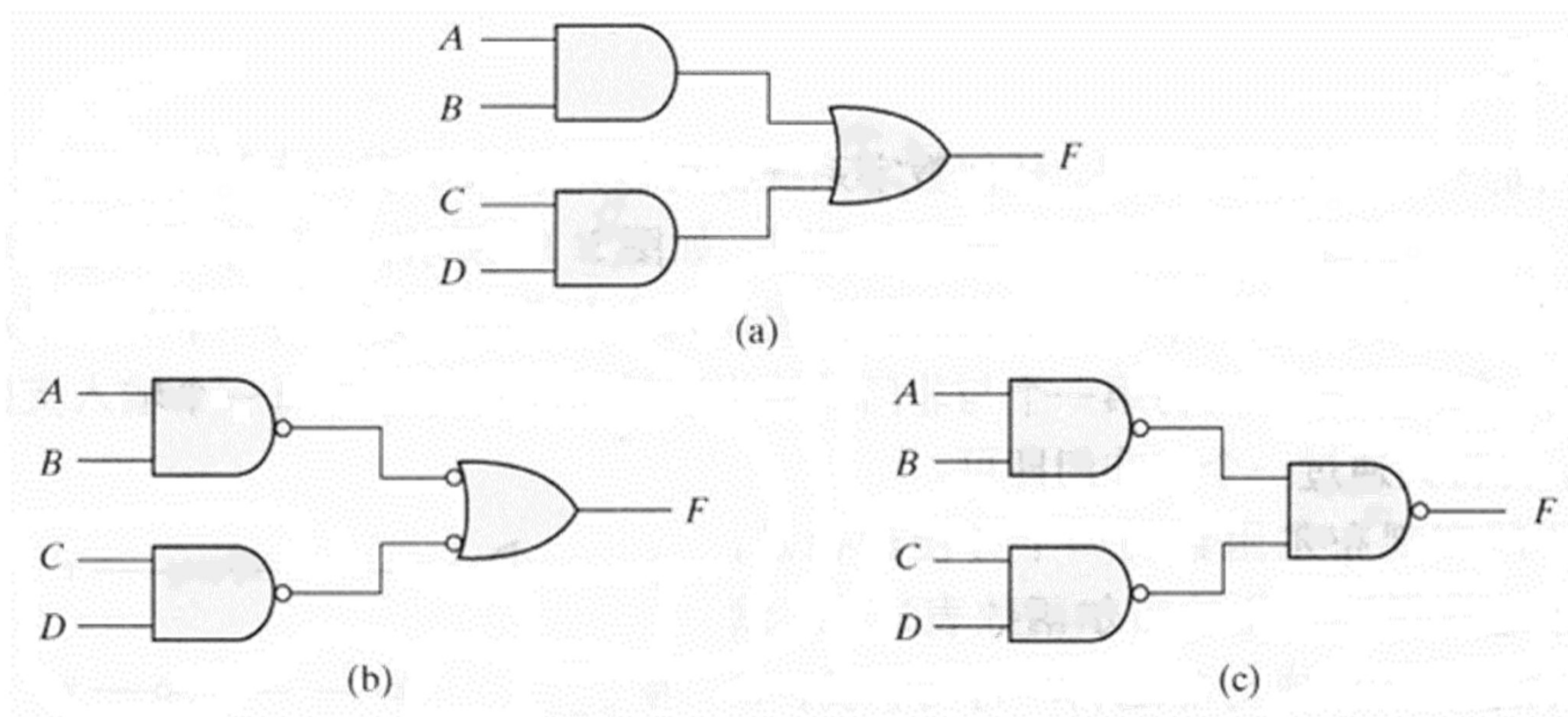


非-或

- 用与非门实现布尔函数，需要将函数表示成积之和式

$$F = AB + CD$$

NAND和NOR实现逻辑函数



$F=AB+CD$ 的三种实现方法

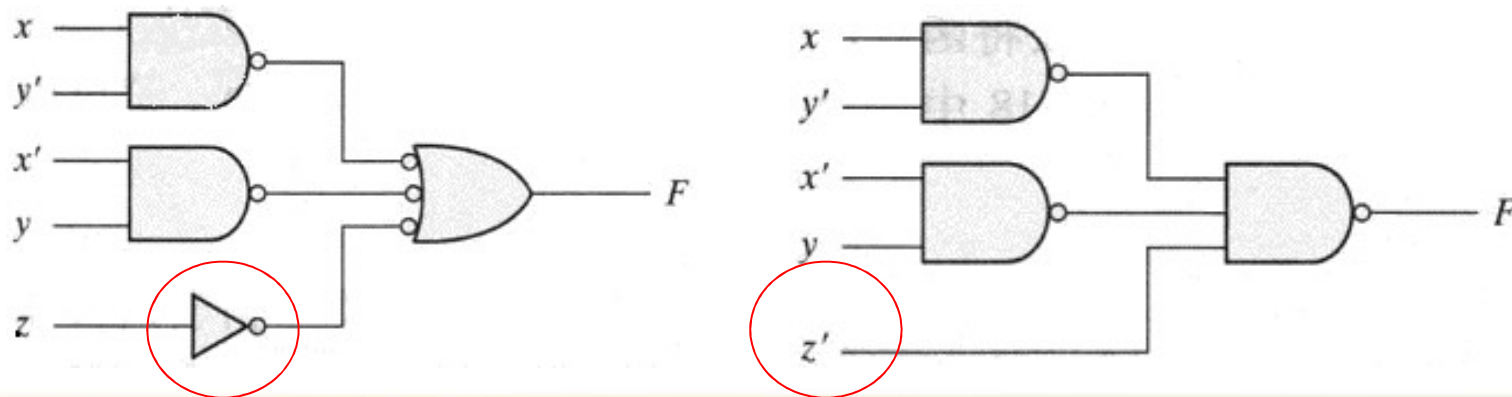
NAND和NOR实现逻辑函数

■ NAND实现布尔函数（续）

- 例：用与非门实现下面的布尔函数：

$$F(x, y, z) = \sum (1, 2, 3, 4, 5, 7)$$

- 1) 化简为积之和式： $F = xy' + x'y + z$
- 2) 采用两级与非门实现



NAND和NOR实现逻辑函数

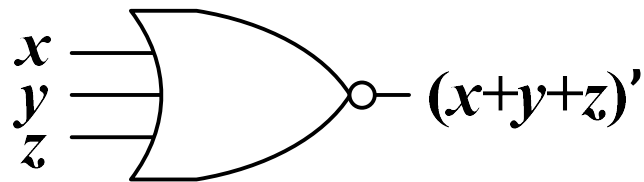
■ NAND实现布尔函数（续）

- 根据布尔函数画出逻辑图的具体步骤：
 - 1) 将函数化简为积之和式
 - 2) 画出函数中至少含有两个字母变量乘积项的与非门电路，每个乘积项的字母变量都是与非门的输入
 - 3) 在第二级画一个单独的门，采用“与-非”或“非-或”图形符号，其输入来自第一级门电路的输出
 - 4) 只有单个字母的项，在第一级需要一个非门，或用反变量

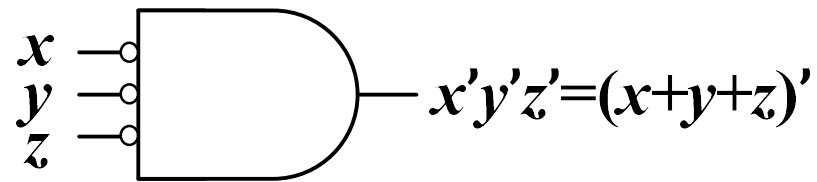
NAND和NOR实现逻辑函数

■ NOR实现布尔函数

- 或非运算是与非运算的对偶，或非运算的所有规则和步骤都是相应的与非逻辑的对偶
- 两种等价的或非门表示



或-非



非-与

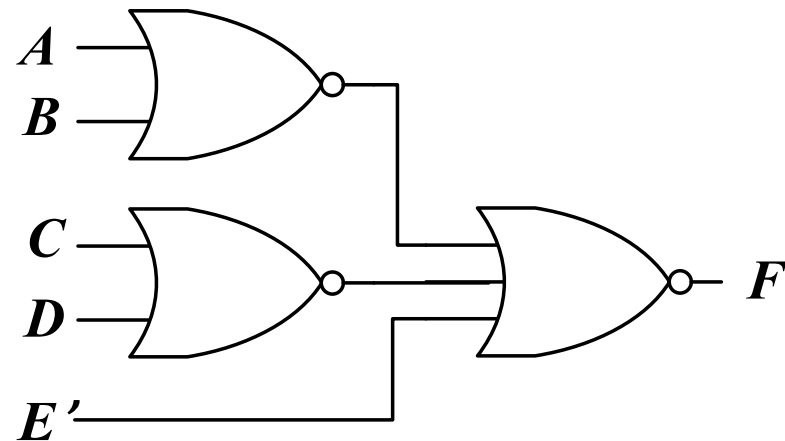
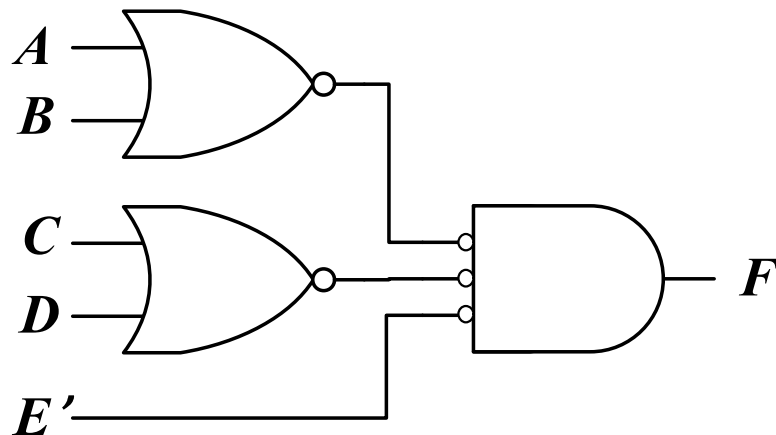
- 用或非门实现布尔函数，需要将函数表示成**和之积式**
第一级门电路用或门实现表达式中的每个和项，第二级电路再用与门实现乘积关系

NAND和NOR实现逻辑函数

■ NOR实现布尔函数（续）

- 例：用或非门实现下面的布尔函数：

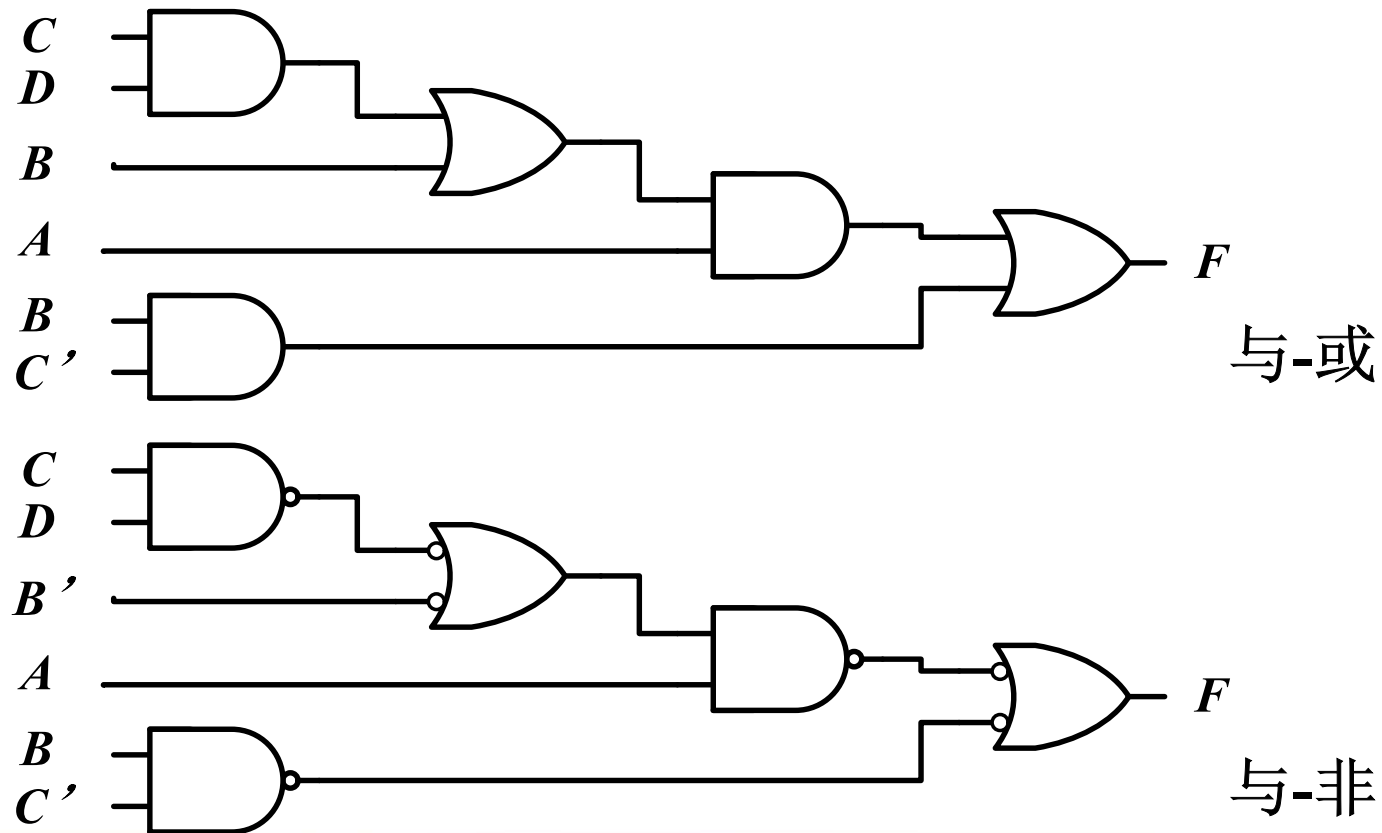
$$F = (A + B)(C + D)E$$



NAND和NOR实现逻辑函数

■ 多级门电路

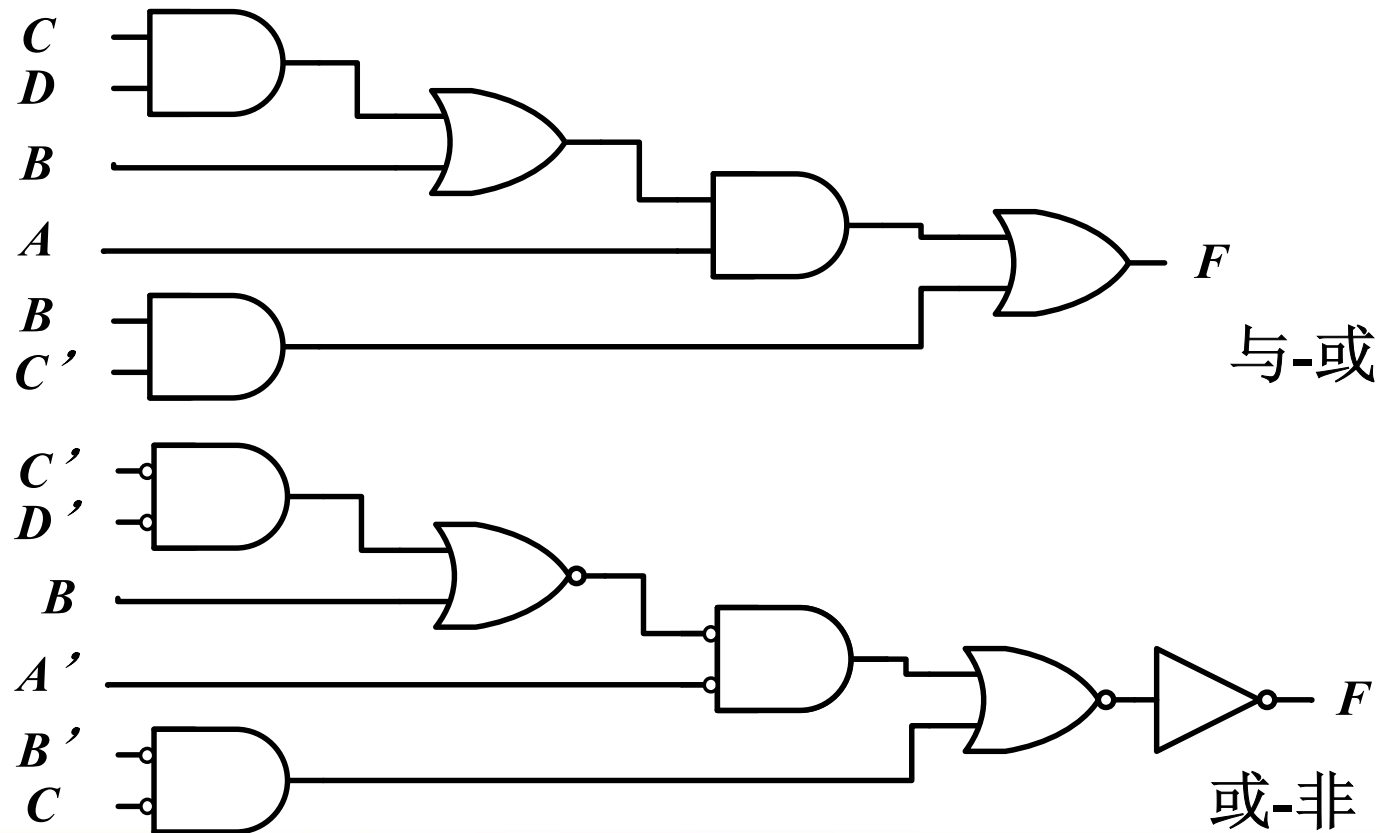
- 采用多级门电路实现： $F = A(CD + B) + BC'$



NAND和NOR实现逻辑函数

■ 多级门电路

- 采用多级门电路实现： $F = A(CD + B) + BC'$



其他两级门电路实现

■ 不可化简形式

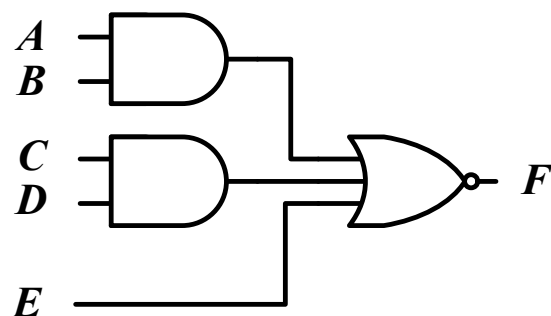
- 四种类型的门（与门、或门、与非门、或非门）分两级电路有**16**种可能的组合方式
- ✓ 8种组合可以被简化为单一的运算，称为可简化形式
- ✓ 其余8种不可简化组合模式用来实现积之和式或和之积式

与-或	或-与
与非-与非	或非-或非
或非-或	与非-与
或-与非	与-或非

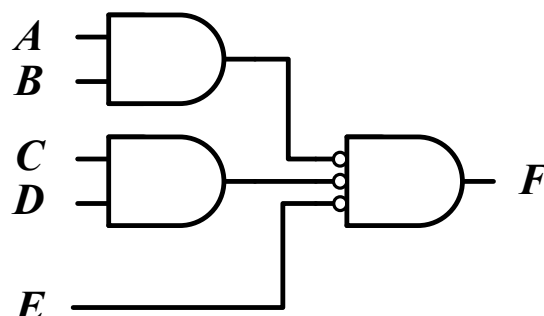
其他两级门电路实现

■ 与-或非形式

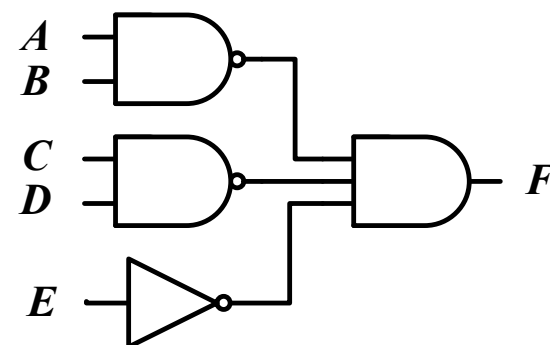
- 与非-与、与-或非是两种等价形式，都能实现与-或非功能



与-或非



与-或非



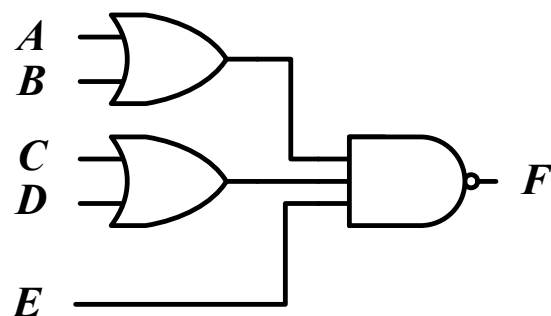
与非-与

与-或非电路: $F = (AB + CD + E)'$

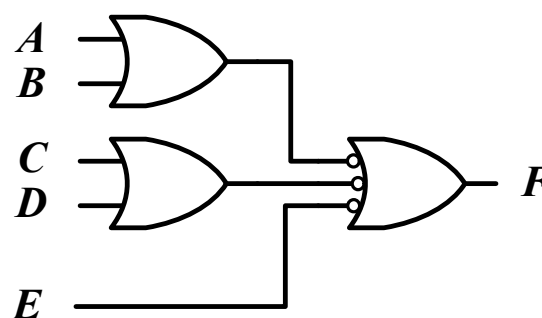
其他两级门电路实现

■ 或-与-非形式

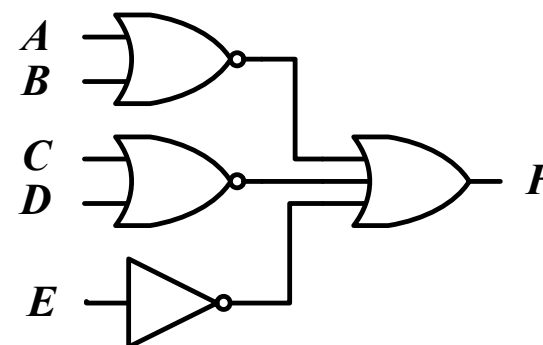
- 或非-或、或-与非也是两种等价形式，都能实现或-与-非功能



或-与非



或-与非



或非-或

或-与非电路: $F = ((A+B)(C+D)E)'$

其他两级门电路实现

■ 基本步骤概括

等价的不可化简模式		实现函数	将 F' 化简为	得到的输出
(a)	(b)			
与-或非	与非-与	与-或-非	卡诺图中合并 0 项, 得到积之和的形式	F
或-与非	或非-或	或-与-非	卡诺图中合并 1 项, 得到和之积式后 再取反	F

其他两级门电路实现

- 例：用表中所列的四种两级电路形式实现图中所示函数

		y			
		yz	00	01	$\overbrace{11 \quad 10}$
x	0	m_0	1	m_1	0
	1	m_4	0	m_5	0
		m_3	0	m_7	0
		m_2	0	m_6	1

$$F = x'y'z' + xyz'$$

$$F' = x'y + xy' + z$$

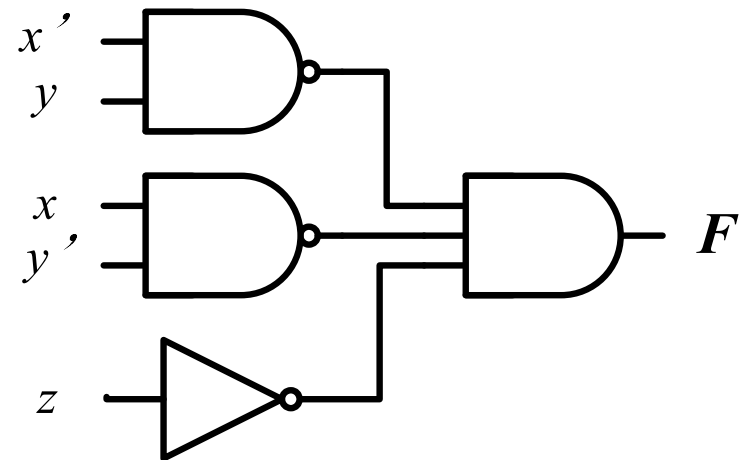
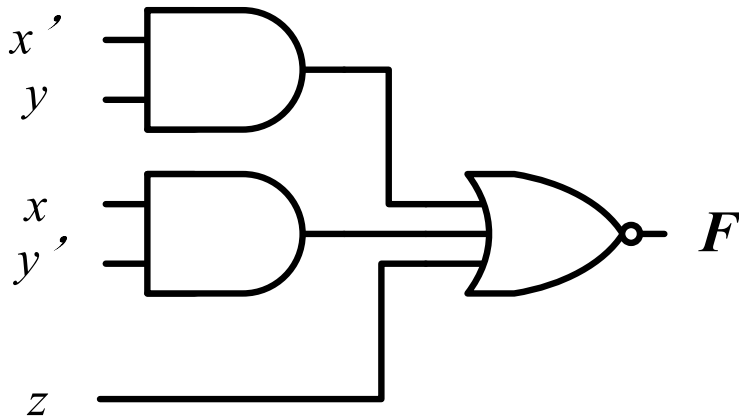
通过合并图中0项，将函数的反函数化简为积之和式

$$F' = x'y + xy' + z$$

则

$$F = (x'y + xy' + z)'$$

其他两级门电路实现



$$F = (x'y + xy' + z)'$$

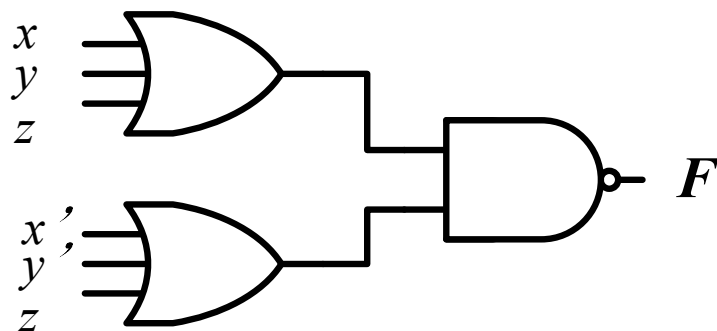
其他两级门电路实现

“或-与非”需要将函数的反函数化简成和之积式。通过合并图中1项，将函数的反函数化简为积之和式

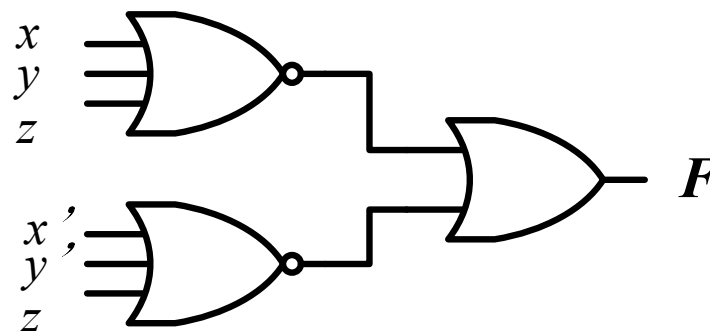
$$F = x'y'z' + xyz'$$

则

$$F' = (x + y + z)(x' + y' + z)$$



或-与非



或非-或

$$F = ((x + y + z)(x' + y' + z))'$$

布尔运算与CMOS逻辑门

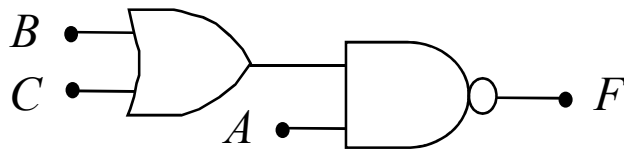
- 二进制数和二进制逻辑
- 布尔函数
- 范式与标准式
- MOSFET开关
- 基本的CMOS逻辑门
- 门电路化简
- CMOS复合逻辑门

CMOS复合逻辑门

构造单个CMOS逻辑门完成复杂逻辑功能

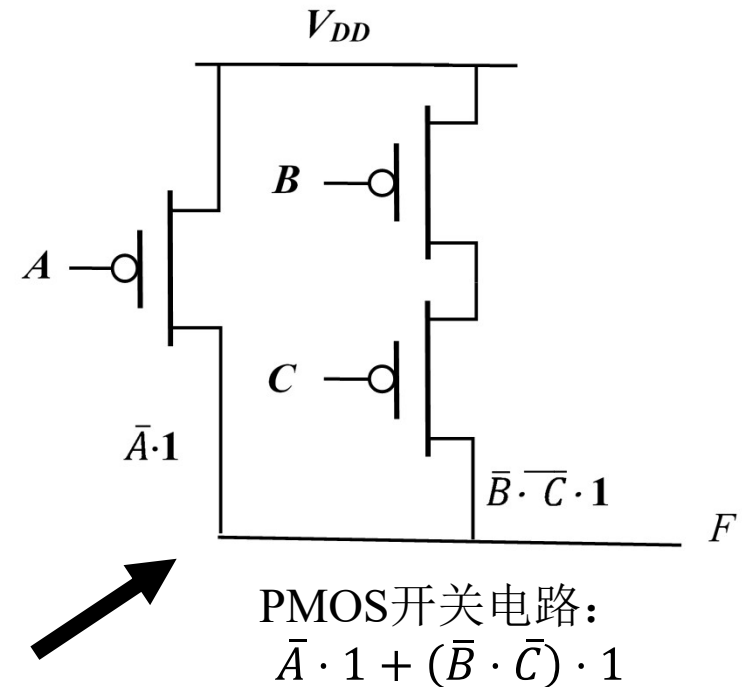
布尔表达式: $F(A, B, C) = \overline{A \cdot (B + C)}$

逻辑电路



对函数 F 运用摩根定理展开得到:

$$\begin{aligned} F(A, B, C) &= \overline{A \cdot (B + C)} \\ &= \overline{A} + \overline{(B + C)} = (\overline{A} + (\overline{B} \cdot \overline{C})) \cdot 1 \\ &= \overline{A} \cdot 1 + (\overline{B} \cdot \overline{C}) \cdot 1 \end{aligned}$$



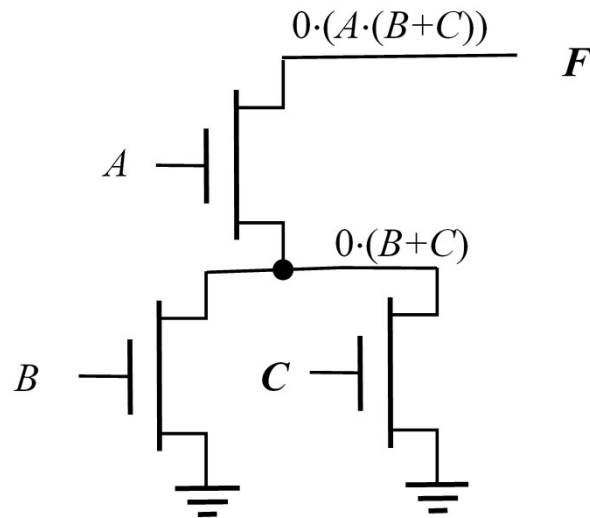
CMOS复合逻辑门

构造单个CMOS逻辑门完成复杂逻辑功能

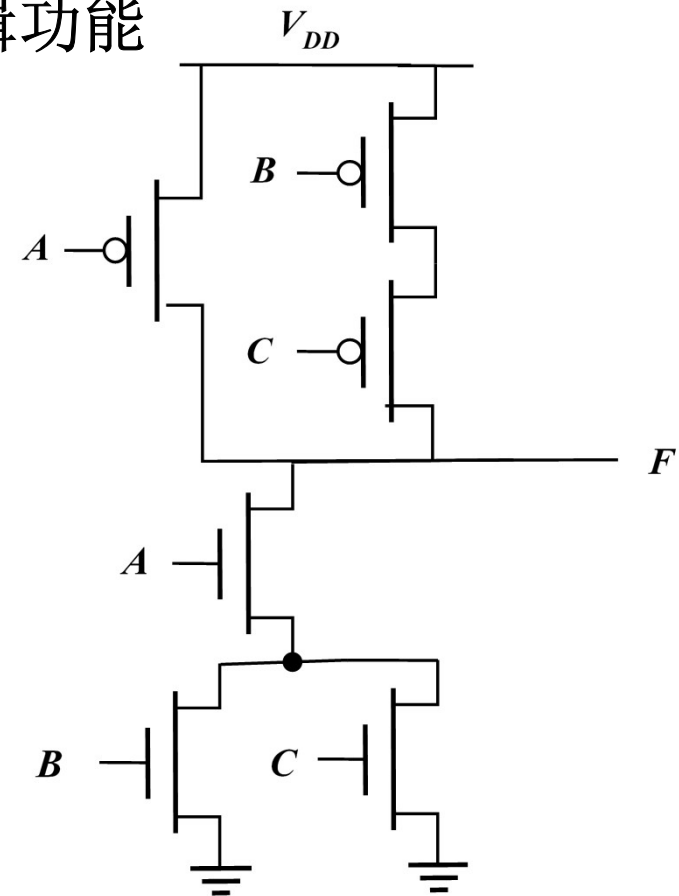
布尔表达式: $F(A, B, C) = \overline{A \cdot (B + C)}$

当 $A=1$ 且 $(B+C)=1$ 时, $F=0$ 。即:

$$0 \cdot (A \cdot (B + C))$$



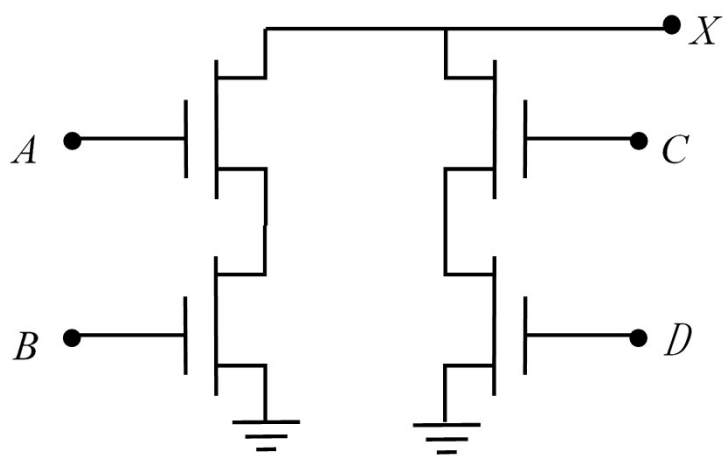
NMOS开关电路: $0 \cdot (A \cdot (B + C))$



互补CMOS复合逻辑门电路:
 $F(A, B, C) = \overline{A \cdot (B + C)}$

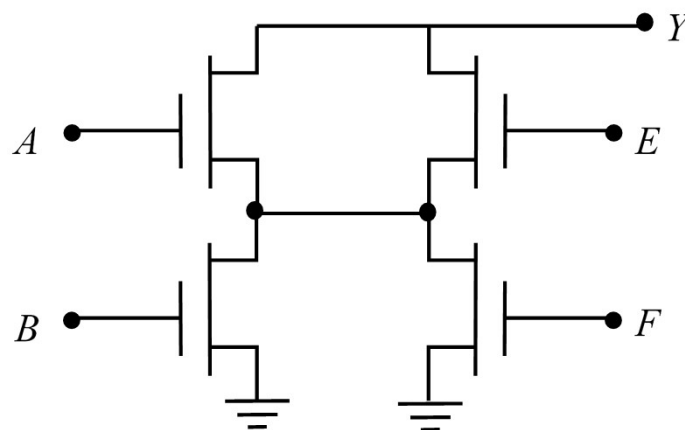
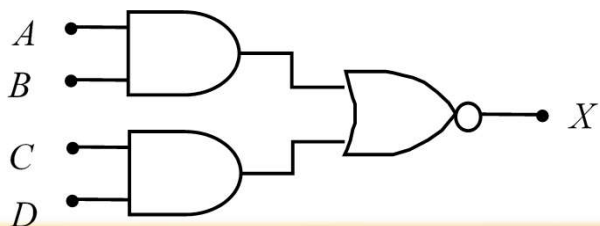
CMOS复合逻辑门

利用NMOS和PMOS串并联规则构造AOI和OAI逻辑电路



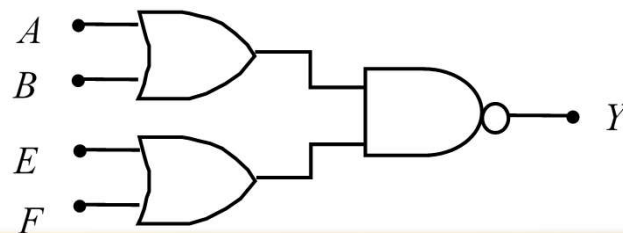
NMOS AOI电路

$$X = \overline{(A \cdot B) + (C \cdot D)}$$



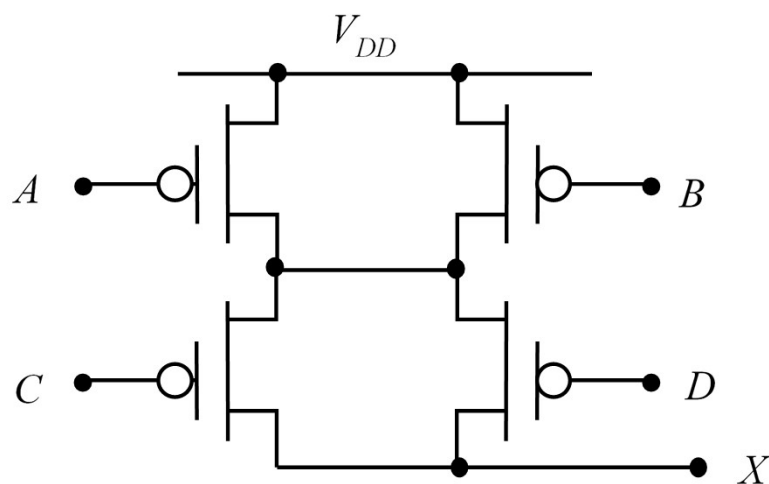
NMOS OAI电路

$$Y = \overline{(A + E) \cdot (B + F)}$$



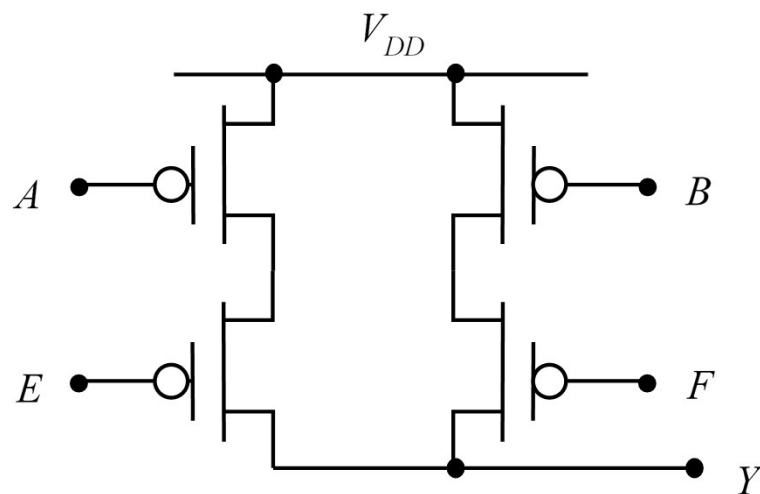
CMOS复合逻辑门

利用NMOS和PMOS串并联规则构造AOI和OAI逻辑电路



PMOS AOI电路

$$X = \overline{(A \cdot B) + (C \cdot D)}$$



PMOS OAI电路

$$Y = \overline{(A + E) \cdot (B + F)}$$

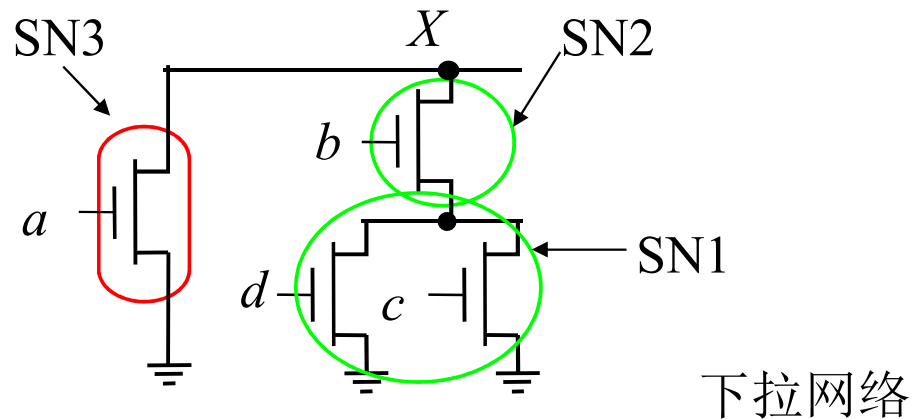
CMOS复合逻辑门

利用**PUN**和**PDN**的对偶性构造复合逻辑门电路

$$X = \overline{a + b \cdot (c + d)}$$

第一步，根据**NMOS**串并联规则构造**PDN**网络：

- 第一组：输入为**c**和**d**的**NMOS**并联；
- 第二组：输入为**b**的一个**NMOS**与第一组串联；
- 第三组：输入为**a**的一个**NMOS**与“第一组-第二组”的电路并联



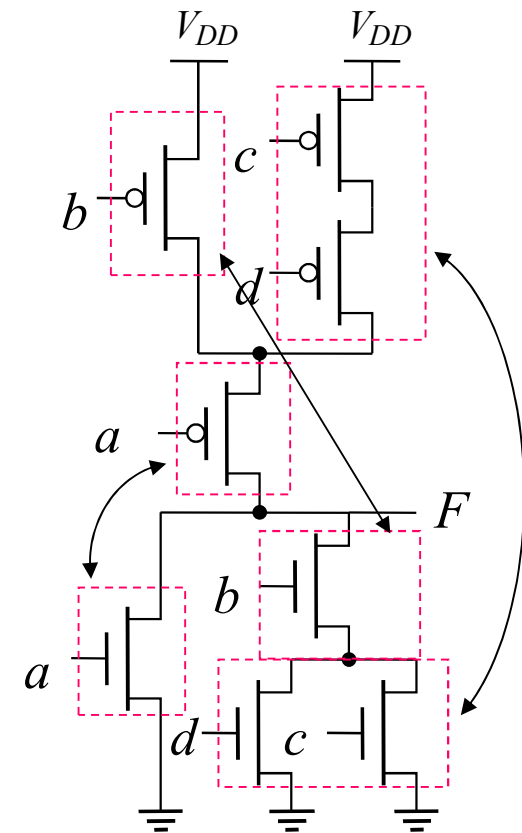
CMOS复合逻辑门

利用**PUN**和**PDN**的对偶性构造复合逻辑门电路

$$X = \overline{a + b \cdot (c + d)}$$

PMOS网络采用与**NMOS**网络对偶的方式构造，每组**PMOS**与具有相同输入的一组**NMOS**相对应，即：

- 第一组：输入为***c***和***d***的**PMOS**串联
- 第二组：输入为***b***的一个**PMOS**与第一组**PMOS**并联
- 第三组：输入为***a***的一个**PMOS**与“第一组-第二组”**PMOS**串联



完整的AOI复合门电路

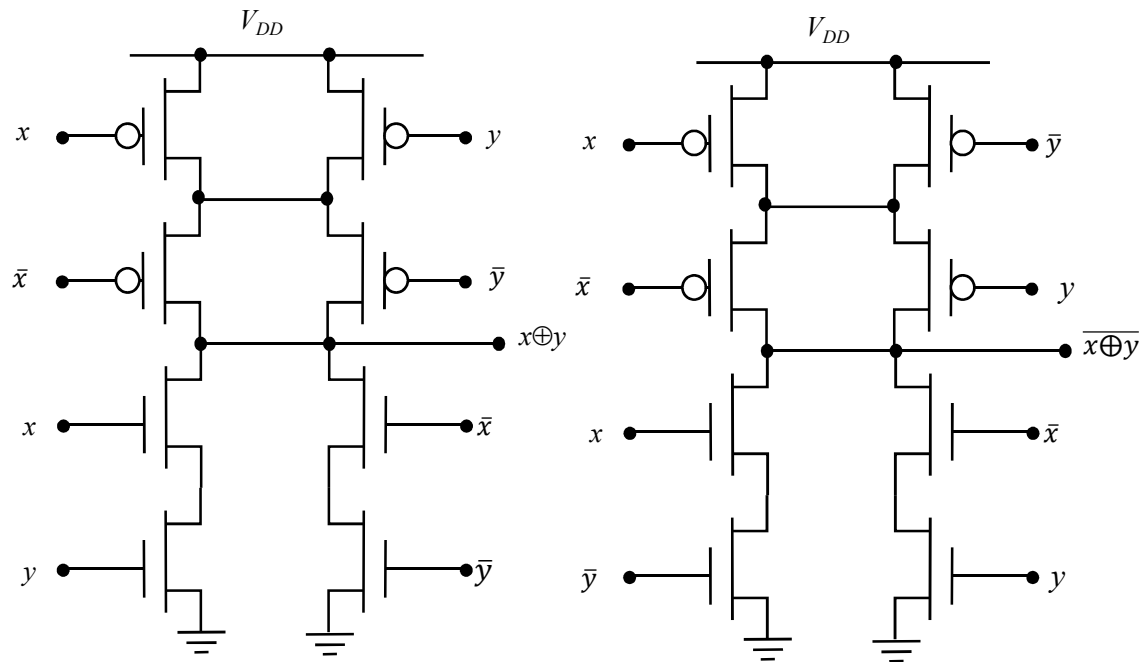
CMOS复合逻辑门

异或（XOR）门和异或非（XNOR）门

$$x \oplus y = x \cdot \bar{y} + \bar{x} \cdot y \quad \text{积之和}$$

$$\overline{x \oplus y} = x \cdot y + \bar{x} \cdot \bar{y}$$

$$x \oplus y = \overline{(x \oplus y)} = \overline{x \cdot y + \bar{x} \cdot \bar{y}} \quad \text{AOI结构}$$



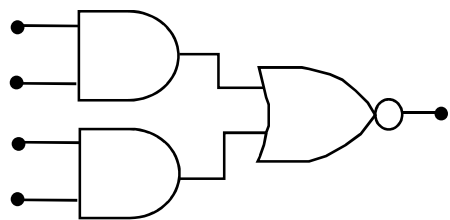
异或（XOR）

异或非（XNOR）

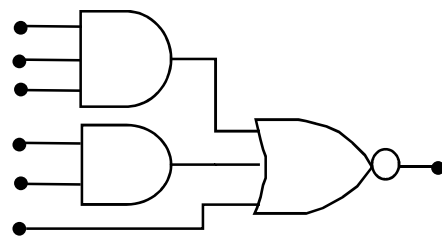
CMOS复合逻辑门

通用AOI和OAI逻辑门

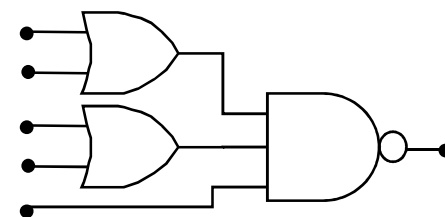
命名方法



AOI22

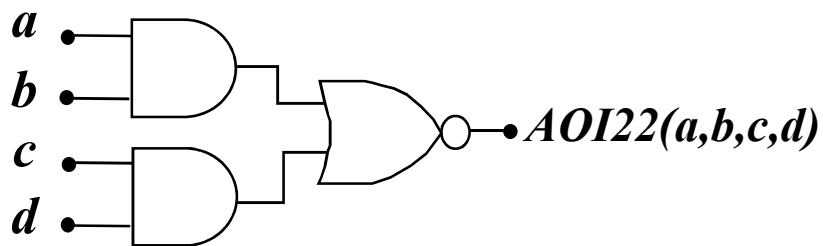


AOI321

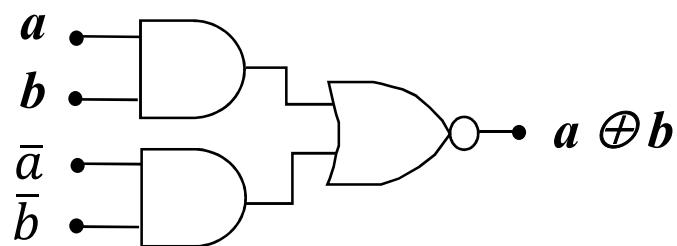


OAI221

利用通用逻辑门建立不同的逻辑功能



$$AOI22(a, b, c, d) = \overline{a \cdot b + c \cdot d}$$



$$a \oplus b = AOI22(a, b, \bar{a}, \bar{b})$$